



OPEN

DATA DESCRIPTOR

A benchmarking framework and dataset for learning to defer in human-AI decision-making

Jean V. Alves¹✉, Diogo Leitão¹, Sérgio Jesus¹, Marco O. P. Sampaio¹, Javier Liébana¹, Pedro Saleiro¹, Mário A. T. Figueiredo^{2,3} & Pedro Bizarro¹

Learning to Defer (L2D) algorithms improve human-AI collaboration by deferring decisions to human experts when they are likely to be more accurate than the AI model. These can be crucial in high-stakes tasks like fraud detection, where false negatives can cost victims their life savings. The primary challenge in training and evaluating these systems is the high cost of acquiring expert predictions, often leading to the use of simplistic simulated expert behavior in benchmarks. We introduce OpenL2D, a framework generating synthetic experts with adjustable decision-making processes and work capacity constraints for more realistic L2D testing. Applied to a public fraud detection dataset, OpenL2D creates the financial fraud alert review dataset (FiFAR), which contains predictions from 50 fraud analysts for 30 K instances. We show that FiFAR's synthetic experts are similar to real experts in metrics such as consistency and inter-expert agreement. Our L2D benchmark reveals that performance rankings of L2D algorithms vary significantly based on the available experts, highlighting the need to consider diverse expert behavior in L2D benchmarking.

Background & Summary

Recently, a growing body of research has focused on human-AI collaboration (HAIC), with several authors arguing that humans and AI have complementary strengths and weaknesses^{1,2}. Collaborative systems have shown that humans can correct model predictions in specific instances¹, and that humans, in collaboration with a Machine Learning (ML) model, can achieve higher performance than either the expert or the model alone³.

The state-of-the-art approach to managing assignments in HAIC is *learning to defer* (L2D)^{4–13}, in which algorithms choose whether to assign a task to a human expert or an ML model, leveraging their complementary strengths. In a multi-expert setting, the algorithm must choose between $J + 1$ decision makers: a human from a team of J experts or the ML model. Training and testing multi-expert L2D algorithms requires every expert's prediction for every instance^{3,12}. To our knowledge, the only public dataset containing human predictions used in multi-expert L2D is the NIH Clinical Center X-ray dataset^{5,13,14}, aimed at detecting airspace opacity; each instance features the predictions of 3 radiologists, sampled from a team of 22. This dataset's main drawback is the rare overlap in the images reviewed by different experts. Hemmer *et al.*⁵ found only four expert pairs with more than 800 shared image predictions, limiting their experiments to 2-radiologist teams. This issue is also observed in CIFAR-10H¹⁵, where the maximum overlap between annotators is 200 instances.

Due to the unavailability of large datasets with concurrent human predictions and the cost of obtaining large amounts of data reliably annotated by human experts, L2D research often uses synthetic “human” decisions. A common approach involves using *label noise* to produce arbitrarily accurate predictions. Various authors^{4,8,11} use CIFAR-10¹⁶, simulating experts with high accuracy on five classes but random accuracy on the remaining five. However, these synthetic experts' decisions depend only on an instance's class, meaning the methods in these benchmarks are not challenged to learn nuanced and varied types of expertise. In line with previous work on learning with noisy labels^{17,18}, we argue that *instance-dependent label noise* (IDN) is more realistic, as human error is likely to depend on each instance's features. Finally, we observe that current L2D research overlooks the limitations of human work capacity, despite these being a challenge in real-world systems^{1,19}, where typically only one expert makes a decision per instance.

¹Feedzai, Coimbra, Portugal. ²Instituto Superior Técnico, ULisboa, Lisboa, Portugal. ³Instituto de Telecomunicações, Lisbon, Portugal. ✉e-mail: jean.alves@feedzai.com

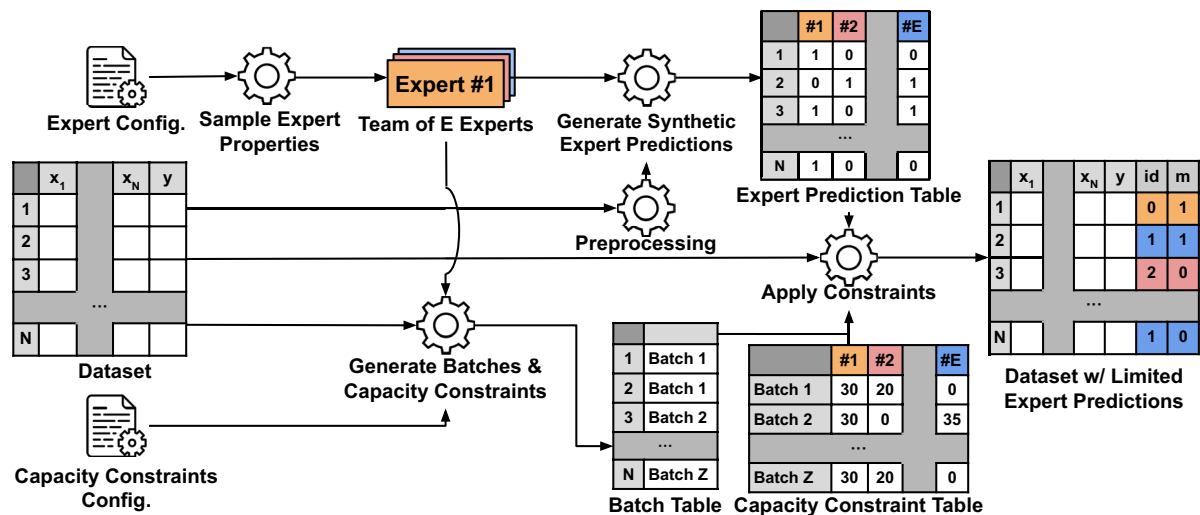


Fig. 1 Generating Synthetic Expert Predictions and Capacity Constraints using the OpenL2D Framework.

To promote thorough benchmarking in L2D research, we present OpenL2D, an open-source framework that allows users to (i) create highly customizable synthetic experts with control over feature dependence, bias towards a protected attribute, and average performance; (ii) generate realistic human work capacity constraints, limiting the number of cases deferred to humans in a given time window; (iii) generate realistic training datasets with limited expert predictions due to user-defined capacity constraints, as well as testing conditions subject to user-defined expert availability and capacity. Therefore, OpenL2D allows for the development and testing of L2D methods under real-world challenges.

We use OpenL2D on a publicly available fraud detection dataset²⁰, generating the FiFAR dataset, comprised of a team of 50 synthetic fraud analysts with realistic decision-making properties. We provide every expert's prediction for every 30 K instances deemed fraudulent by a fraud detection model, thus simulating an “alert-review” scenario, where experts are tasked with reviewing high-risk bank account opening applications. We validate our experts' decision-making processes by analyzing their performance and key metrics such as inter and intra-rater agreement, comparing them to current literature on high-stakes decision-making. We also compare the decision process of our simulated experts with that of real fraud analysts, demonstrating their similarities. By simulating these experts' work capacity constraints, we generate a set of training scenarios where each instance only has the prediction of one expert, simulating more realistic data availability. Finally, we test two capacity-constraint-aware L2D methods by training them in these scenarios and testing them under varied capacity constraints, demonstrating that L2D algorithms' performance ranking is significantly affected by the subset of humans available in both training and testing. This highlights the importance of considering complex expert decision-making processes in L2D.

Although FiFAR is designed to be used out-of-the-box, OpenL2D allows users to tweak FiFAR's experts and their capacity constraints to suit their benchmarks and generate complex synthetic experts in binary classification tabular datasets, allowing for increased variety and complexity in L2D benchmarking.

Methods

Synthetic Data Generation Framework - OpenL2D. We now present our framework, which is represented in Fig. 1. OpenL2D takes as an input a tabular dataset for a binary classification task, represented on the center left of the diagram. OpenL2D generates the supplementary data necessary to allow for thorough L2D research in said binary tabular dataset, including synthetic expert predictions, as well as training and testing conditions based on the work capacity constraints of the expert team. The process begins by sampling the properties of each expert in an E expert team, based on a set of user-defined properties. Then, it generates the expert prediction table, which is the set of predictions of every expert for every instance in the dataset, represented in the top right. This set of predictions can optionally be used to develop and evaluate L2D methods that require every expert's prediction to exist for every instance. Optionally, the user may define a set of work capacity constraints, to be applied over batches of instances, represented in the bottom right. The capacity constraints limit the number of predictions that can be made by a given expert over a specified number of cases, representing a given time window. These can be used to simulate a training dataset with limited expert predictions (one prediction per instance), constrained by expert work capacity, represented in the rightmost part of the diagram, where the columns “m” and “id” represent the set of expert predictions and the corresponding identifiers, respectively. If an assignment system can be configured to assign under capacity restrictions, the user may also generate capacity constraints to be applied in testing.

Dataset. Our framework is designed to work with on any tabular dataset for binary classification, where each instance i is represented by a feature vector x_i and the label given by $y_i \in \{0, 1\}$. To use the framework, the user

must define the categorical and numerical features present in their dataset, as well as the column corresponding to the label. Should the classification task at hand entail fairness concerns, the user may also define which features correspond to protected attributes, allowing for simulation of experts with bias spanning across multiple attributes, such as gender and race. Finally, if the user wishes to simulate experts who have access to additional information, such as an ML classifier's score, the user may specify the column corresponding to it.

Synthetic Expert Generation. In order to generate our synthetic experts' predictions, we resort to an *instance-dependent noise* (IDN) approach, similar to previous work^{17,18} on learning with noisy labels. We generate the prediction $m_{j,i}$ of synthetic expert j on instance i by flipping said instance's label y_i with probability $\mathbb{P}(m_{j,i} \neq y_i | \mathbf{x}_i, y_i)$. We aim to accommodate scenarios with significant class imbalance (e.g., medicine, financial fraud), as well as cost-sensitive tasks, where the cost of misclassification for false positives (FP) and false negatives (FN) differs (e.g., in medicine, false alarms are generally considered less harmful than failing to diagnose a disease). To do so, we separately simulate the probability of type I and type II errors. This introduces a dependence on the label but allows for better individual control of FP and FN misclassifications.

In some real-world applications of HAIC^{1,21}, an ML model is trained to perform the same classification task as the human expert. In these collaborative systems, the model score predicted for a given instance may be shown to the expert, together with other relevant information. It has been shown that the expert's performance is significantly impacted by seeing the ML model's score along with the rest of the information pertaining to each instance^{1,21,22}.

We define the expert's probabilities of error, for a given instance, as a function of a pre-processed version of its features $\bar{\mathbf{x}}_i$ and an optional model score $M(\mathbf{x}_i)$, so that the scale of each feature's values does not impact their relative importance. The probabilities of error are computed as

$$\begin{cases} \mathbb{P}(m_{j,i} = 1 | y_i = 0, \mathbf{x}_i) = \sigma \left(\beta_0 + \alpha \frac{\mathbf{w} \cdot \bar{\mathbf{x}}_i + w_M M(\mathbf{x}_i)}{\sqrt{\|\mathbf{w}\|^2 + w_M^2}} \right) \\ \mathbb{P}(m_{j,i} = 0 | y_i = 1, \mathbf{x}_i) = \sigma \left(\beta_1 - \alpha \frac{\mathbf{w} \cdot \bar{\mathbf{x}}_i + w_M M(\mathbf{x}_i)}{\sqrt{\|\mathbf{w}\|^2 + w_M^2}} \right), \end{cases} \quad (1)$$

where σ denotes a sigmoid function. Each expert's probabilities for type I and type II errors are parameterized by: β_0 , β_1 , α , $\mathbf{w}$, and w_M . These parameters are sampled, for each individual expert, from user-defined distributions, generating a group of experts with similar properties.

The weight vector $\mathbf{w}$ embodies a relation between the features and the probability of error. The weights are shared in both formulas, with opposite signs, ensuring that if a feature contributes to increase the FP probability, it will, in turn, decrease the FN probability. We choose to enforce this restriction in order to produce a more intuitive decision-making process: if an expert does more FPs in a given region of the feature space, we also expect them to do fewer FNs, assuming they will be biased towards classifying more instances as positive in said region. Each component w_i of this vector is independently sampled from a user-defined distribution. To introduce bias against a protected attribute (*i.e.*, age or gender), the user may separately define the distribution of the weight vector component w_p , where p denotes the index of the protected attribute in the feature vector. The optional model score weight w_M allows the user to impose a dependence on the model score of an independent ML model. We normalize the feature weights so that we can separately control, with parameter α , the overall magnitude of the variation of the probability of error due to the features.

The values of β_0 and β_1 are, respectively, related to the false positive rate (FPR) and false negative rate (FNR) of a given expert. The user can define the distributions of FPR and FNR within an expert group, or, alternatively, the distribution of the expected misclassification cost of the expert's predictions, by providing the costs of FP and FN errors. This allows for full control of the expert's average performance, based on commonly used metrics. Our framework then calculates the values of β_1 and β_0 that yield the sampled FNR and FPR, or the sampled expected misclassification cost.

Various groups of experts can be generated within a team, by setting the sampling distributions within each group. This allows users to create more varied behaviour within a team, for example, by creating a group of experts with a high dependence on the model score ($w_M \sim \mathcal{N}(\mu, \sigma)$, with $|\mu| \gg 1$), and a group with low dependence on the model score ($w_M \sim \mathcal{N}(\mu, \sigma)$, with $|\mu| \ll 1$). A more detailed description of the function and sampling process of each parameter follows. Details on how each of these parameters are defined within the framework's configuration files can be found in our repository, referenced in the Code availability Section.

Feature dependence weights generation. Our framework provides two possible methods for generating the feature weights for a group of experts. The user may manually define the distribution for each feature i , so that $w_i \sim \mathcal{N}(\mu_i, \sigma_i)$. Alternatively, we allow the user to define the desired distribution to be shared across all features, by defining the parameters of a "spike-and-slab" distribution²³. A spike-and-slab prior is a generative model in which a random variable u either attains some fixed value v , called the *spike*, or is drawn from another prior distribution p_{slab} , called the *slab*. In our case, we set $v = 0$. That is, u is either zero, or drawn from the slab density $\mathcal{N}(\mu_w, \sigma_w)$, where μ_w , σ_w are defined by the user. To sample the values of w_i , we first sample a Bernoulli latent variable $Z \sim \text{Ber}(\theta)$ to select if w_i is sampled from the *spike* or the *slab*. If $Z = 0$, w_i attains the fixed value $v = 0$, if $Z = 1$, w_i is drawn from the slab density. As such, the spike-and-slab prior induces sparsity unless $\theta = 1$,

allowing for the generation of experts whose probabilities of error are swayed by a varying number of features, by changing the value of θ . The distribution of $w_M \sim \mathcal{N}(\mu_M, \sigma_M)$ can be defined separately to control the expert's model-dependency. Should there be one or more protected attributes, the user may define $\mu_{p_1}, \sigma_{p_1}, \dots, \mu_{p_n}, \sigma_{p_n}$, in order to impose fairness/unfairness on the simulated experts' predictions. With these two options, users can easily define a variety of feature dependencies across their expert team.

Controlling variability and expert's consistency. While the weight vector controls the relative influence that each feature has on the probability of error, the parameter α , in turn, controls the global magnitude of this influence. For $\alpha = 0$, the probability of error is the same for all instances within a class, which is equivalent to *class-dependent* label noise, as used in previous L2D testing^{4,8,11}. In turn, for very large α , the probability would saturate at the extremes of the range of the sigmoid function, 0 or 1, resulting in a deterministic decision-making process. The value of α can therefore be seen as a measure of the consistency, or intra-rater reliability, of a decision maker, as larger values of α will result in experts who are more likely to produce the same decisions when faced with similar inputs. The value of α is sampled from the user defined distribution $\alpha \sim \mathcal{N}(\mu_\alpha, \sigma_\alpha)$.

Controlling expert performance. In binary classification, two metrics often used to evaluate performance are the false positive rate (FPR) and false negative rate (FNR). Our framework allows users to set the distributions from which expert j 's target performance metrics are sampled, $T_{FPR_j} \sim \mathcal{N}(\mu_{FPR}, \sigma_{FPR})$ and $T_{FNR_j} \sim \mathcal{N}(\mu_{FNR}, \sigma_{FNR})$. Alternatively, the user may wish to measure expert j 's performance by using the expected misclassification cost $\mathbb{E}[C]_j$ resulting from their decisions. In this case, the user would define the distribution from which the target expected misclassification cost is sampled $T_{\mathbb{E}[C]_j} \sim \mathcal{N}(\mu_C, \sigma_C)$. We now describe how the values of β_1 and β_0 are calculated in order to achieve the desired performance in both of these cases, starting by assuming that the user wishes to define $T_{\mathbb{E}[C]_j}$.

In a binary task, the misclassification cost for an instance is either 0, if the prediction is correct, or given by the cost of a false positive c_{FP} or a false negative c_{FN} mistake. Therefore, the expected misclassification cost per instance is given by

$$\begin{aligned}\mathbb{E}[C]_j &= \mathbb{E}_{(x,y,m_j)}[c_{FP}\mathbb{I}(m_j = 1 \wedge y = 0) + c_{FN}\mathbb{I}(m_j = 0 \wedge y = 1)] \\ &= c_{FN}\mathbb{E}_{(x,y,m_j)}[\lambda\mathbb{I}(m_j = 1 \wedge y = 0) + \mathbb{I}(m_j = 0 \wedge y = 1)],\end{aligned}\quad (2)$$

where $\lambda = c_{FP}/c_{FN}$. As division by a constant factor does not affect the ranking of different sets of assignments, λ represents the only degree of freedom in the cost structure of the classification task at hand. As such, without loss of generality, we set $c_{FN} = 1$ and $c_{FP} = \lambda$. The expected cost resulting from an expert's predictions can be empirically approximated by using a partition of the dataset with N instances

$$\mathbb{E}[C]_j \approx \frac{1}{N} \sum_{i=1}^N [\lambda \mathbb{I}(m_{j,i} = 1 \wedge y_i = 0)] + \frac{1}{N} \sum_{i=1}^N [\mathbb{I}(m_{j,i} = 0 \wedge y_i = 1)] = \lambda FPR_j \frac{N_0}{N} + FNR_j \frac{N_1}{N}, \quad (3)$$

where FPR_j and FNR_j represent, respectively, the empirical estimates of the FPR and FNR of expert j , and N_0 and N_1 are the numbers of samples in class 1 and class 0 respectively. Assuming that the values of w , α and w_M are set, $FPR_j = FPR_j(\beta_0)$ and $FNR_j = FNR_j(\beta_1)$. There is a degree of freedom in this equation: if we choose the value of either β_0 or β_1 , the other must attain a specific value so that we obtain $\mathbb{E}[C]_j = T_{\mathbb{E}[C]_j}$. Inverting the previous expression we obtain

$$FPR_j = \frac{NT_{\mathbb{E}[C]_j}}{\lambda N_0} - \frac{N_1}{\lambda N_0} FNR_j \quad (4)$$

meaning that any pair of target values for FPR_j and FNR_j that obey the above equation will yield $T_{\mathbb{E}[C]_j}$. We allow the user to set $T_{\mathbb{E}[C]_j} \sim \mathcal{N}(\mu_C, \sigma_C)$, and to also set an upper bound and lower bound for the costs. To sample FPR_j and FNR_j in order to achieve $T_{\mathbb{E}[C]_j}$, we sample a random value of T_{FNR_j} , and calculate T_{FPR_j} according to the above expression, ensuring both values belong to the interval (0, 1). As such, both methods of defining the expert performance rely on establishing the value of T_{FNR_j} and T_{FPR_j} , either directly or indirectly. We can now calculate the value of β_0 and β_1 to obtain the desired T_{FNR_j} and T_{FPR_j} . From Equation (1),

$$FPR_j(\beta_1) = \frac{1}{N} \sum_i \sigma\left(\beta_1 + \alpha \frac{w \cdot x_i}{\|w\|}\right). \quad (5)$$

where, for notational simplicity we set $w_M = 0$, but the result is similar for $w_M \neq 0$. It is then possible to show that the function $FPR_j(\beta_1)$ is monotonically increasing, as it is well known that the sigmoid is monotonically increasing. Since the function $FPR_j(\beta_1)$ is monotonically increasing, and strictly bounded to the interval [0, 1], then, for any target false positive rate T_{FPR_j} , then the equation

$$\text{FPR}_j(\beta_{1_j}) - T_{\text{FPR}_j} = 0 \text{ for } T_{\text{FPR}} \in (0, 1). \quad (6)$$

has an unique solution, which we obtain using a bisection method²⁴. A similar reasoning applies for β_{0_j} and T_{FNR_j} . Finally, we can control each expert's FPR and FNR by solving these equations for β_{1_j} and β_{0_j} . As a partition of the dataset is used to calculate the empirical approximations of $\mathbb{P}(y = 1)$ and $\mathbb{P}(y = 0)$, as well as FPR_j and FNR_j , deviations from the target performance metrics may occur in the rest of the dataset due to temporal distribution shifts.

Feature preprocessing. For our simulation of experts, the feature space is transformed as follows. Numeric features are transformed to quantile values, and shifted by -0.5 , resulting in features with values in $[-0.5, 0.5]$. This ensures that the features impact the probability of error independently of their original scale. Categorical features are target-encoded, that is, encoded into non-negative integers by ascending order of target prevalence. These values are divided by the number of categories, so that they belong to the $[0,1]$ interval, and shifted so that they have zero mean.

Capacity constraints. Since most real-world applications of HAIC are limited by work capacity constraints, L2D evaluation benchmarks should include them. Humans are not able to process instances at the same speed as ML models, being limited in the number of cases they may process on any given time period (e.g., work day). A real-world assignment system must then process instances taking into account the human limitations over a given "batch" of cases, corresponding to a pre-defined time period. The user may divide the dataset into several batches and, for each batch, define the maximum number of instances that can be processed by each expert. For a dataset of N instances, capacity constraints can be represented by a vector $\mathbf{b}$, where component b_i denotes to which batch instance $i \in \{1, \dots, N\}$ belongs, as well as a human capacity matrix H , where element $H_{b,j}$ is a non-negative integer denoting the number of instances expert j can process in batch b . To define the capacity constraints, the user must set the following parameters:

- **Batch Size:** denotes the number of cases in each batch. Different batch sizes allow for testing assignment methods over long and short horizons. To allow for the generation of different batches with the same size, the user may set the seed for the distribution of cases throughout the batches.
- **Deferral Rate:** represents the fraction of each batch that can be deferred to the expert team.
- **Distribution:** a binary flag {'Homogeneous', 'Variable'}. Should the distribution be homogeneous, every expert has the same capacity. Otherwise, each expert's capacity is sampled from $\mathcal{N}(\mu_d = \text{Deferral_Rate} \times \text{Batch_Size} / N_{\text{experts}}, \sigma_d \times \mu_d)$, with user-defined σ_d . This distribution is chosen so that each expert's sampled capacity fluctuates around the value corresponding to an homogeneous distribution. We set the standard deviation as $\sigma_d \times \mu_d$ so that the user can define the capacity variability as a proportion of μ_d . The user may set the seed for this sampling, allowing for different configurations with the same variability.
- **Number of Experts:** Number of Experts: the number of experts available in each batch. This allows for testing several different group configurations without generating new experts, or to generate scenarios where not all experts work in the same time periods, simulating workers with varying shifts. The user may set the seed for the sampling of the available experts, to generate, for example, multiple scenarios where the only change is the sample of available experts, enabling users to test the robustness of L2D systems to changes in expert availability.

Generating training scenarios. In most decision-making tasks where a team of humans is involved, due to the limited human work capacity, each instance is reviewed by a single human worker¹ or by a small fraction of a larger team²⁵. If we were to consider the predictions of an expert team gathered over the course of their employment (i.e., keeping track of social worker's decision¹) we would not have the necessary data to train most L2D methods. This means that the data needed to train an L2D system would have to be purposefully collected for the development of said system, which would entail high costs. L2D research should then focus on developing algorithms assuming the existence of scarce human predictions, that is, a single expert's prediction per instance.

In order to generate such a training scenario, a user needs to have completed the steps previously described, generating the synthetic expert predictions as well as a set of capacity constraints to be applied (see also Fig. 1). Finally, the user can distribute the cases in the training environment throughout the experts according to a given function (i.e., random assignment), while respecting their capacity constraints. This creates a training set for the development of assignment systems with limited expert predictions, a key barrier to real-world implementation of L2D algorithms.

Generation process of FiFAR dataset. We now describe how we used OpenL2D to create a team of 50 complex and varied synthetic fraud analysts, in order to supplement a public financial fraud detection dataset without human predictions, thus generating the *Financial Fraud Alert Review* (FiFAR) dataset.

Dataset and use case. We use the first version of the base variant of the publicly available bank-account-fraud tabular dataset (<https://www.kaggle.com/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/1?select=Base.csv>) introduced by Jesus *et al.*²⁰. This dataset is sizeable (one million rows) and has other key properties that are relevant for our use case and common across many high-stakes decision-making applications. Each instance

represents a bank account opening application, in which the features contain information about the application and the applicant. Each instance has a label that denotes if the instance is a fraudulent application (1) or a legitimate application (0). The task of a decision maker (automated or human) when reviewing a bank account opening application is to predict whether or not the application is fraudulent. If the decision-maker deems the application fraudulent, it is rejected; otherwise, it is accepted. As such, the generated expert predictions will follow the same labeling convention, 1 if the expert deems the application fraudulent and chooses to reject it, and 0 otherwise.

This data was generated by leveraging state-of-the-art Generative Adversarial Network (GAN) models on an anonymized, real-world bank account opening fraud detection dataset. Consequently, this dataset poses challenges common in real-world high-stakes applications. Firstly, there is a high class imbalance, with a fraud prevalence of 1.1% over the entire dataset. Secondly, this dataset poses a cost-sensitive problem, where the cost of a false positive (declining a legitimate event) must be weighed against the cost of a false negative (accepting a fraudulent event). Note also that a positive prediction (predicted fraud) results in a declined bank account application. Consequently, false positives in account opening fraud can significantly hinder a person's life, by denying possibility to open a bank account or access credit. ML models trained on this dataset without measures to preserve fairness tend to be biased and raise more false fraud alerts (false positive errors) for older clients (≥ 50 years), thus reducing their access to a bank account. Therefore, this task entails a relevant fairness concern and is appropriate for evaluating the fairness of not only ML models, but also assignment systems. Finally, the dataset contains changes in the data distribution over time, often referred to as concept drift²⁶, which can severely impact the predictive performance of ML models.

Task. The optimization objective stated in the BAF dataset²⁰ follows a Neyman-Pearson criterion: maximize the fraud detection recall at 5% FPR. This establishes an implicit relationship between the costs of a false positive and a false negative error. In a cost sensitive task, our optimization goal is to obtain a set of predictions $\hat{Y}$ that minimize the expected misclassification cost $\mathbb{E}[C]$. In Section Controlling Expert Performance, we demonstrate that the expected misclassification cost can be calculated by having access to a single parameter λ . As such, all that remains is to establish a relationship between the Neyman-Pearson criterion and the value of λ . From classical Bayes decision theory²⁷, we can establish a relationship between the threshold of a binary classifier and the misclassification costs. For a given instance, the ideal classification is the one that minimizes the posterior expected loss. As such, the optimal prediction for any given instance $\mathbf{x}_i$ is 1 if and only if the expected cost of predicting 1 is no larger than the expected cost of predicting 0, i.e.,

$$(1 - p)c_{FP} \leq pc_{FN}, \quad (7)$$

where $p = P(y = 1 | \mathbf{x}_i)$ is the posterior probability that $\mathbf{x}_i$ belongs to the positive class. An estimate of p is given by our “Alert Model”, in the form of the model score output for a given instance. By equalizing both sides of the expression, we determine the optimal value of p above which we should predict the positive class, that is, the value of the model's threshold t . This value is known, as the optimal threshold t for our Alert Model was adjusted such that the Neyman-Pearson criterion is met (i.e., it is the threshold that yields 5% FPR).

$$(1 - t)c_{FP} = tc_{FN} \Leftrightarrow \lambda = \frac{t}{1 - t} = 0.057 \quad (8)$$

This value will be used when generating our synthetic experts.

Alert review setting. There are several possible ways for ML models and humans to cooperate. In previous L2D research, it is common that any instance can be deferred to either the model or the expert team. However, in real-world settings, due to limitations in human work capacity and legal restrictions, it is common to use an ML model to screen instances, raising alerts that are then reviewed by human experts^{1,28}. In an alert review setting, humans only predict on a fraction of the feature space, that is, the instances flagged by the “Alert Model”. We will train a L2D system to work in tandem with the Alert Model, by deferring the alerts in an intelligent manner.

To set up this scenario, we divide the eight months of the BAF dataset according to Fig. 2. The splits are made across different months in the BAF dataset as there are changes in data distribution across months, a core challenge for ML approaches.

The Alert Model is trained on months 1–3 and validated on month 4. This time frame was chosen as limiting the model's training to months 1–2 and validation on month 3 would result in a significantly worse performance. However, generating the expert predictions starting only on month 5 would use only 2 months for training the L2D systems. As such, we chose to generate the human expert predictions, which are influenced by the Alert Model, in the period from month 4 to month 8. We allow for this overlap as the Alert Model primarily serves the purpose of lending realism to the scenario, by screening the instances on which the human experts will make predictions, as well as by influencing their decisions via the model score assigned to each instance. As it is not an active component in any L2D system that may be trained on this data, we found this approach to be the best compromise between realism and data availability.

By splitting the dataset in this way, we obtain a total of three months for L2D system training, with one month for validation and the last one for testing.

Alert model. We train the Alert Model to predict the fraud label on the first three months of the dataset, validating its performance on the fourth month. We use the LightGBM¹⁹ algorithm, due to its proven high performance on tabular data^{29,30}. The model is trained by minimizing binary cross-entropy loss. The hyperparameter selection process is detailed in Table 1. This model achieves a recall of 57.9% in validation, for a threshold

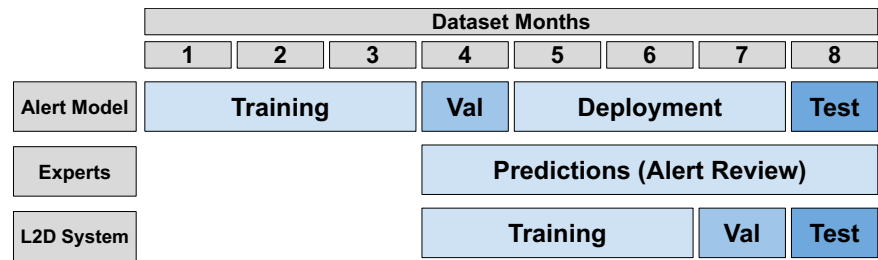


Fig. 2 Data splits used for Alert Model and L2D system training.

Hyperparameter	Values or Interval	Distribution	Selected
boosting_type	“goss”		“goss”
enable_bundle	False		False
n_estimators	[50,5000]	Log	94
max_depth	[2,20]	Unif.	2
num_leaves	[10,1000]	Log	145
min_child_samples	[5,500]	Log	59
learning_rate	[0.01, 0.5]	Log	0.3031421
reg_alpha	[0.0001, 0.1]	Log	0.0012637
reg_lambda	[0.0001, 0.1]	Log	0.0017007

Table 1. Alert Model: LightGBM hyperparameter search space. At each trial, hyperparameter sampling is conducted via the Tree-structure Parzen Estimator (TPE) algorithm, as implemented in the Optuna⁴¹ package. The optimization objective is to maximize Recall at 5% FPR in validation (month 4). Optimization is run for 100 rounds with early-stopping after 10 rounds without improvement in the optimization objective. The total time for hyperparameter optimization was 13 minutes using 25 cores of an Intel(R) Xeon(R) Gold 5120 CPU @ 2.20 GHz.

$t = 0.050969$, defined to obtain a 5% FPR in validation. In Fig. 3 we present the ROC curve for the Alert Model, calculated in months 4–8.

Expert instantiation. In this section, we detail the parameter selection for the generation of the team of 50 synthetic fraud analysts. We chose to create a total of 50 experts as this far surpasses the numbers previously considered in L2D research^{4–13}. As such, users interested in conducting research with smaller expert teams can easily select a sub-sample of our already generated experts. Furthermore, a large number of experts allows users to test the robustness of L2D systems to changes in the expert team (i.e., teams that work on different shifts), by sub-sampling different, smaller sets of experts. Should 50 experts not be sufficient, generation of larger numbers of experts with the same properties is done easily by resorting to the code available in our repository (see Section Code availability).

The properties of our simulated team’s decisions, as well as the reasoning behind the selection of the expert’s parameters, are evaluated in detail in the Technical Validation Section, where we will assess the validity of the FiFAR dataset by analysing how our experts compare to previous literature on human decision-making processes, cognitive-biases, and high-stakes decision-making performance. We also compare the synthetic decision-making process to that of real-world fraud analysts.

Our team of synthetic experts aims to simulate real-world behavior, by ensuring that our experts exhibit properties that have been documented in high-stakes decision-making. To select the distribution of the α parameter (see Equation (1)), we conducted a series of experiments in order to ensure that the intra-rater consistency was similar to that of real-world experts, selecting $\mu_\alpha = 15$, $\sigma_\alpha = 3.5$. For more information, see subsection “Internal Consistency of Human decision-making Processes”.

We define the expert’s cost distribution by assuming that the analyst’s performance is better than simply rejecting all the alerted applications ($E[C] = 0.0496$). We choose $\mu_c = 0.035$, $\sigma = 0.005$, setting an upper bound for the expert cost at $\text{Top}_c = 0.046$ and the lower bound at $\text{Bottom}_c = 0.02$. We also impose bounds for the FPR and FNR of experts, ensuring that $\text{FNR}_j \in [0.01, 1]$ and $\text{FPR}_j \in [0.01, 1]$, in order to guarantee that the label flipping process produces both types of errors for all experts. For more information, see subsection “Expert Performance Distribution”.

When generating the weight vector $\mathbf{w}$, our objective is to create a team of experts with a reasonable level of inter-rater agreement, meaning that the decision-processes in an expert team should be relatively similar, but not identical. To generate a realistic decision-making process, we first determine the correlation ρ between the dataset’s features and the ground-truth label. To quantify the correlation between the label and categorical features, we use Cramér’s V, which is a measure of association between two variables, producing a value between 0 and 1. We use Pearson’s correlation coefficient to quantify the correlation between the label and continuous numerical

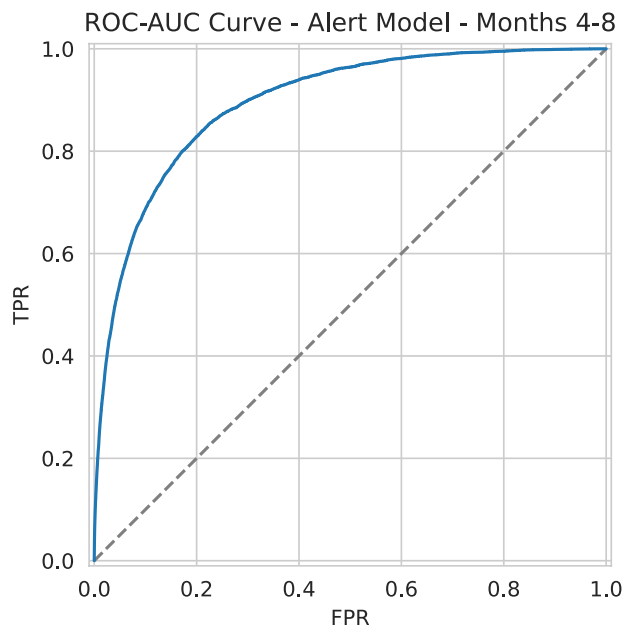


Fig. 3 ROC-Curve - Alert Model - Months 4–8.

features. We then select the 15 features with the highest correlation with the label (either negative or positive), and set their respective $\mu = \rho$, $\sigma = 0.1$, making each feature's influence on the probability of error scale according to their correlation with the label, but introducing variations in the decision-making processes of different experts. All other feature weights are sampled from $\mathcal{N}(0, 0.05)$, in order to introduce further variation in the decision-making processes, but guaranteeing that the experts are not too dissimilar. In this way, we create a set of relatively similar decision-making processes allowing for the possibility of some outliers (see Subsections “Inter-Rater Agreement” and “Feature Dependence and Algorithmic Bias”). The values pertaining to the distribution of the model score weight w_M were manually chosen to produce a dependency on the model score similar to real-world fraud analysts (see Subsection “Feature Dependence and Algorithmic Bias”). The distribution of the protected attribute's weight w_p was also manually defined, chosen so that the experts exhibit varying levels of bias against older customer's applications (see subsection “Unfairness Against Social Groups”). The values of μ and σ for each of the feature weight distributions used to instantiate our expert team are available in Table 2.

Note that a positive sign for the Pearson's correlation coefficient will result in the probability of a FP rising as the value of the numerical feature rises, while a negative sign will produce the opposite effect. This means that the generated experts will tend to produce false positives/negatives if a feature's value is positively/negatively correlated with the label. As the Crámer's V only produces positive values belonging to the $[0,1]$ interval, we cannot draw conclusions on the direction of the correlation. However, taking into account that the categorical features are target encoded in the data pre-processing, all weights are made positive, so that the nominal values for which the fraud prevalence is higher produce a higher likelihood of a false positive.

Training scenarios. Our dataset includes 25 constrained training scenarios with only one expert prediction per instance. All scenarios aim to simulate a period from months 4 to 7 (see Fig. 2), where a random deferral system is deployed, meaning that the alerted applications are randomly deferred to a team of experts. This creates a realistic dataset of expert predictions resulting from collecting a single prediction per instance, as is often the case in real-world systems^{1,19}. These scenarios were generated by setting the “Distribution” as homogeneous, the “Deferral Rate” to 1. To introduce variability across scenarios with the purpose of testing L2D systems' robustness to changes in the available experts, we create 5 distinct teams of 10 experts by setting the “Number of Experts” parameter to 10 and using 5 distinct values for the expert availability seed. The number 10 was chosen to allow for sufficient variability within teams. Combining these 5 different teams with 5 different seeds for the distribution of cases throughout batches, we generate a total of 25 total training scenarios. Month 8 is left for testing, on the subset of instances flagged by the Alert Model in that time period.

Testing scenarios. The Testing Scenarios are defined for each of the five teams, as we assume that the team of experts available in training is the same team that will be queried in testing (see Section L2D Benchmarking). As such, the five instances of an L2D method trained in the five training scenarios pertaining to a given team will be tested in the testing scenarios pertaining to said team.

For each team, five different capacity constraints are imposed in testing: one homogeneous and four variable with $\sigma_d = 0.2$. The number of experts for these capacity constraints are sampled by using the same $n_experts_seed$ values defined in training. This guarantees that the sampled expert teams are the same in testing as in training.

feature	μ	σ
income	0.326	0.1
name_email_similarity	−0.388	0.1
prev_address_months_count	0	0.05
current_address_months_count	0	0.05
customer_age	0.07	0.02
days_since_request	0	0.05
intended_balcon_amount	0	0.05
payment_type	0.362	0.1
zip_count_4w	0	0.05
velocity_6h	−0.41	0.1
velocity_24h	0	0.05
velocity_4w	−0.447	0.1
bank_branch_count_8w	0	0.05
date_of_birth_distinct_emails_4w	−0.342	0.1
employment_status	0	0.05
credit_risk_score	0.626	0.1
email_is_free	0.383	0.1
housing_status	0	0.05
phone_home_valid	−0.347	0.1
phone_mobile_valid	0	0.05
bank_months_count	0	0.05
has_other_cards	0	0.05
proposed_credit_limit	0.735	0.1
foreign_request	0	0.05
source	0	0.05
session_length_in_minutes	0	0.05
device_os	0.354	0.1
keep_alive_session	−0.44	0.1
device_distinct_emails_8w	0.378	0.1
device_fraud_count	0	0.05
model_score	0.7	0.3

Table 2. Expert Feature Weight Distributions.

In a real-world scenario, the capacity of the classifier would be orders of magnitude larger than that of the human experts, as the cost of querying a model is much lower than that of querying a human expert. Furthermore, we could expect the classifier to flexibly stand in for absent humans, should an expert, for any reason, not be able to predict on a given batch, for example. In our testing scenarios, however, we aim to measure the ability of L2D algorithms to intelligently assign, by identifying the most appropriate decision maker for a given instance. To better assess these capabilities, we choose to define rigid capacity constraints, demanding that each expert be deferred, on average, an equal number of instances, in order to ensure that the assignment system is able to make the most of each member of the human-AI team. For all capacity constraints, the deferral rate, that is, the fraction of instances that are deferred to the 10 expert team, is set to 10/11, to ensure that, on average, each decision maker (each of the 10 humans or the classifier) predicts on 1/11 of the instances.

Data Records

The FiFAR dataset³¹ has been made available for download through the figshare platform (see citation for DOI link). The structure of the dataset is represented in Fig. 4. The dataset's root directory, "FiFAR", contains several sub-folders corresponding to separate components of our dataset. Over the following subsections, we detail the purpose of each directory and the files contained therein.

alert_data. The "alert_data" directory contains the file "Base.csv", corresponding to the first version of the base variant of the BAF dataset (<https://www.kaggle.com/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/1?select=Base.csv>)²⁰. This file is included to allow users to easily access the original version of the dataset without any of the processing steps detailed in section [Generation Process of FiFAR dataset](#). This file contains one million rows, each representing a feature-engineered bank account application with the following fields:

- **income** (numeric): Annual income of the applicant (in decile form). Ranges between [0.1, 0.9]
- **name_email_similarity** (numeric): Metric of similarity between email and applicant's name. Higher values represent higher similarity. Ranges between [0, 1].

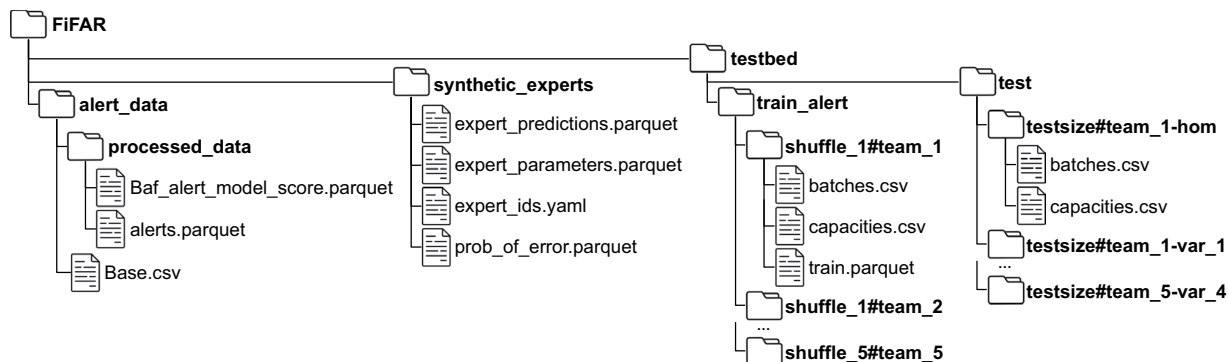


Fig. 4 Folder Structure of the FiFAR Dataset.

- **prev_address_months_count** (numeric): Number of months in previous registered address of the applicant, i.e. the applicant's previous residence, if applicable. Ranges between $[-1, 380]$ months (-1 represents a missing value).
- **current_address_months_count** (numeric): Months in currently registered address of the applicant. Ranges between $[-1, 429]$ months (-1 is a missing value).
- **customer_age** (numeric): Applicant's age in years, rounded to the decade. Ranges between $[10, 90]$ years.
- **days_since_request** (numeric): Number of days passed since application was done. Ranges between $[0, 79]$ days.
- **intended_balcon_amount** (numeric): Initial transferred amount for application. Ranges between $[-16, 114]$ (negatives are missing values).
- **payment_type** (categorical): Credit payment plan type. 5 possible (anonymized) values.
- **zip_count_4w** (numeric): Number of applications within same zip code in last 4 weeks. Ranges between $[1, 6830]$.
- **velocity_6h** (numeric): Velocity of total applications made in last 6 hours i.e., average number of applications per hour in the last 6 hours. Ranges between $[-175, 16818]$.
- **velocity_24h** (numeric): Velocity of total applications made in last 24 hours i.e., average number of applications per hour in the last 24 hours. Ranges between $[1297, 9586]$.
- **velocity_4w** (numeric): Velocity of total applications made in last 4 weeks, i.e., average number of applications per hour in the last 4 weeks. Ranges between $[2825, 7020]$.
- **bank_branch_count_8w** (numeric): Number of total applications in the selected bank branch in last 8 weeks. Ranges between $[0, 2404]$.
- **date_of_birth_distinct_emails_4w** (numeric): Number of emails for applicants with same date of birth in last 4 weeks. Ranges between $[0, 39]$.
- **employment_status** (categorical): Employment status of the applicant. 7 possible (anonymized) values.
- **credit_risk_score** (numeric): Internal score of application risk. Ranges between $[-191, 389]$.
- **email_is_free** (binary): Domain of application email (either free or paid).
- **housing_status** (categorical): Current residential status for applicant. 7 possible (anonymized) values.
- **phone_home_valid** (binary): Validity of provided home phone.
- **phone_mobile_valid** (binary): Validity of provided mobile phone.
- **bank_months_count** (numeric): How old is previous account (if held) in months. Ranges between $[-1, 32]$ months (-1 is a missing value).
- **has_other_cards** (binary): If applicant has other cards from the same banking company.
- **proposed_credit_limit** (numeric): Applicant's proposed credit limit. Ranges between $[200, 2000]$.
- **foreign_request** (binary): If origin country of request is different from bank's country.
- **source** (categorical): Online source of application. Either browser (INTERNET) or app (TELEAPP).
- **session_length_in_minutes** (numeric): Length of user session in banking website in minutes. Ranges between $[-1, 107]$ minutes (-1 is a missing value).
- **device_os** (categorical): Operative system of device that made request. Possible values are: Windows, macOS, Linux, X11, or other.
- **keep_alive_session** (binary): User option on session logout.
- **device_distinct_emails** (numeric): Number of distinct emails in banking website from the used device in last 8 weeks. Ranges between $[-1, 2]$ emails (-1 is a missing value).
- **device_fraud_count** (numeric): Number of fraudulent applications with used device. Ranges between $[0, 1]$.
- **month** (numeric): Month where the application was made. Ranges between $[0, 7]$ (this value is not used in training the Alert Model).
- **fraud_bool** (binary): If the application is fraudulent or not. This is the label for the classification task at hand.

The sub-directory "processed_data", we include the file "BAF_alert_model_score.parquet", which contains the entirety of the instances pertaining to months 4–8 of the original "Base.csv" (months 1–3 were used in training), accompanied by two additional columns:

- **case_id** (numeric): an unique identifier of the bank account opening application. Ranges from [397039, 999999]
- **model_score** (numeric): the Alert Model's score, a measure of the probability that each instance is fraudulent. Ranges between [0,1]

This file allows users to analyze the performance of the Alert Model, and, should they wish to do so, replicate our experiments with a different Alert Model threshold.

Finally, the file “alerts.parquet” contains the subset of instances flagged by the Alert Model, that is, the instances with a ‘model_score’ value above the threshold determined in [Alert Model](#), calculated to ensure maximal recall at 5% FPR. This file contains the instances that the simulated experts will handle, constituting the input dataset fed into the OpenL2D framework for the generation of the synthetic experts.

synthetic_experts. This folder contains the data pertaining to the generated synthetic experts, as well as key information regarding their generation process, and, consequently, their decision-making properties.

The file “expert_predictions.parquet” contains every expert's prediction for every single alerted instance. Each row in this table contains the application identifier ‘case_id’, and, for each of the 50 generated experts, a field named ‘standard#{n}’, where $n \in 0, \dots, 49$, containing the predictions of the corresponding expert. The predictions of each expert are binary in nature, with each expert having predicted that the instance is fraudulent (1) or legitimate (0). This file allows users to model each experts behavior under the restrictive requirement of access to every expert's prediction for each instance, a requirement of many L2D systems, as discussed in [Section Background Summary](#).

In the file “expert_parameters.parquet”, we store the value of each expert's set of parameters, which were sampled in the data generation process detailed in [Section Synthetic Expert Generation](#). This file thus allows users to have access to each of the parameters involved in the expert generation process. This table contains the indexes ‘standard#{n}’, where $n \in 0, \dots, 49$, corresponding to each expert. The fields in each row are named after each of the fields of the ‘Base.csv’ file, previously detailed in the subsection [alert_data](#), with the exclusion of the ‘month’ and ‘fraud_bool’ fields. Each of these fields contain the value corresponding to each component of the feature weight vector $\mathbf{w}$, described in [Section Synthetic Expert Generation](#), thus fully describing the feature dependency of each expert's probability of error (see [Equation 1](#)). Additionally this table contains the following fields related to the expert generation process:

- **model_score** (numeric): the value of the weight given to the Alert Model's score (see w_m in [Equation 1](#))
- **fp_beta** (numeric): the value of the β_0 parameter in [Equation 1](#)
- **fn_beta** (numeric): the value of the β_1 parameter in [Equation 1](#)
- **alpha** (numeric): the value of the α parameter in [Equation 1](#).

The file “prob_of_error.parquet” is a table containing, for each entry identified by the ‘case_id’ value, the probability of false positive and false negative error for each expert $n \in 0, \dots, 49$, calculated according to [Equation 1](#). These values are contained in fields named ‘p_fp_standard#{n}’ and ‘p_fn_standard#{n}’, respectively, allowing users to analyze the properties of each expert's probability of error distribution, which can be useful when studying an L2D method's calibration, for example.

testbed. This folder contains the files pertaining to the benchmarking resources described in the [Sections Training Scenarios](#) and [Testing Scenarios](#), designed for robust evaluation of the performance of L2D methods. These are divided into two sub-directories.

train_alert. The “train_alert” folder contains a sub-folder for each of the 25 training scenarios. As mentioned in [Section Training Scenarios](#), each scenario represents a simulated period of time where a random deferral system is used, thus creating a realistic expert prediction set, resulting from the collection of a single prediction per instance. A total of 25 scenarios are generated by combining 5 different distributions of cases throughout batches with 5 different expert teams, comprised of 10 experts each. Each scenario can thus be identified by its batch shuffle and the available expert team. Consequently, each sub-folder is named ‘shuffle_{i}#team_{j}’, where $i \in 1, \dots, 5$ represents each of the 5 different distribution of cases throughout batches, and $j \in 1, \dots, 5$ represents each of the 5 different expert teams. Within a given scenario's sub-folder, there are three different files.

The train.parquet file corresponds to the collection of expert predictions, obtained through random deferral subject to capacity constraints, as detailed in [Section Training Scenarios](#). This file contains the instances pertaining to months 4–7 of the file ‘alerts.parquet’, described in the Subsection [alert data](#). To each bank account opening application, the following two fields are added: ‘assignment’, which contains the identifier of the expert the application was deferred to, in the format ‘standard#{i}’; and “decision”, which contains expert i 's prediction for said instance.

The ‘batches.csv’ file contains a table where each row represents a bank account opening application, with the fields ‘case_id’, an integer value representing the application's identifier; and ‘batch_id’, an integer value corresponding the batch said application belongs to.

The ‘capacities.csv’ file contains one row per batch, with the fields ‘batch_id’, an integer identifying the batch; and, for each of the ten experts in the team, a ‘standard#{i}’ field, containing an integer corresponding to the maximum number of cases a expert i can process within that batch.

L2D methods developed under our proposed framework (*i.e.*, a single expert prediction per instance, with work capacity constrained deferral) should take as input the ‘train.parquet’ file only, as it simulates realistic data availability.

test. The directory “test” contains a set of 25 sub-directories pertaining to each of the testing scenarios. As detailed in Section [Testing Scenarios](#), there is a total of 5 testing scenarios created for each team considered in testing. Each of these 5 testing scenarios is characterized by different capacity constraints for each of the 10 experts in the team: one homogeneous scenario where each expert can predict on the same number of instances and four different variable capacity scenarios. For each of these scenarios, due to the limited size of the test split (*i.e.* instances pertaining to month 8 in the file ‘alerts.parquet’, described in the Subsection [alert data](#)), the entirety of the test set was considered as a single batch of size 4457. Consequently, each sub-folder is named ‘testsize#team_{i}-hom’ or ‘testsize#team_{i}-var_{j}’, where $i \in 1, \dots, 5$ represents each of the 5 different each of the 5 different expert teams and $j \in 1, \dots, 4$ represents each of the 4 variable capacity scenarios. Within a given scenario’s sub-folder, there are two different files, following the same format for capacity and batch definition followed in section [train alert](#).

The ‘batches.csv’ file contains a table where each row represents a bank account opening application, with the fields ‘case_id’, an integer value representing the application’s identifier; and ‘batch_id’, an integer value corresponding to the batch said application belongs to, which, in our testing scenarios is always 1.

The ‘capacities.csv’ file contains a single row, with the fields ‘batch_id’, which is always 1, thus identifying the single test batch; and, for each of the ten experts in the team, a ‘standard#{i}’ field, containing an integer corresponding to the maximum number of cases an expert i can process within that batch.

L2D methods developed under our proposed framework (*i.e.*, a single expert prediction per instance, with work capacity constrained deferral) should take as input, at time of deferral, the ‘batches.csv’ and ‘capacities.csv’ files, in order to determine the optimal deferral under capacity constraints.

Technical Validation

The validation of our data comprises two parts. First, in Section [Realism of Synthetic Experts and Applicability in L2D](#), we compare the decision-making properties of the generated experts with previous literature on human decision-making in high-stakes scenarios^{21,25,28,32–36}. We also analyze a private dataset of real transaction fraud analysts to gain insights specific to the fraud detection domain.

Second, in Section [L2D Benchmarking](#), we conduct a benchmark of multi-expert L2D algorithms which are compatible with individually defined expert capacity constraints, demonstrating how FiFAR can be used, and how the inclusion of complex expert behaviour and varied testing scenarios can significantly affect the outcome of L2D benchmarks.

Realism of Synthetic Experts and Applicability in L2D. In this section, we evaluate the properties of our synthetically generated experts and how they compare to the existing literature regarding real-world experts in high-stakes decision-making tasks. We will focus on literature regarding “Internal Consistency of Human Decision-making Processes” (intra-rater agreement)^{32,33}, “Inter-Rater Agreement” (agreement across experts)³³, the “Expert Performance Distribution” within a team²⁵, and the potential for “Unfairness Against Social Groups”^{34–36}. We also discuss how these properties can potentially affect L2D systems, and are therefore worthy of consideration when designing L2D benchmarks.

The primary goal of this section is to not only validate the realism of FiFAR’s synthetic fraud analysts, but also to demonstrate how the OpenL2D framework is able to realistically capture properties of human decision-making which are observed across several domains (e.g. medicine^{25,33}, finance^{21,28}, and criminal law³⁶). When possible, we resort to literature specific to the fraud prevention domain^{21,28}.

Finally, in Section [Feature Dependence and Algorithmic Bias](#), we use real-world fraud analysts’ decisions extracted from a private dataset on a different type of fraud (transaction fraud), to derive decision-making properties specific to fraud detection analysts, comparing the properties observed in real analysts to those of our simulated experts. Note that this private dataset was not used in any of the previously mentioned sections as it contains only a single expert’s prediction per instance, making it impossible to calculate inter and intra-rater agreement metrics. Furthermore, the dataset contains missing ground-truth labels, thus making it impossible to evaluate expert performance distributions. However, this data is extremely useful to evaluate how real-world experts’ decisions depend on an instance’s features as well as an auxiliary model score shown to them, a common practice in fraud prevention^{21,28}.

We encourage practitioners aiming to use the OpenL2D framework on datasets in other domains to follow a similar procedure to bolster the realism of their generated experts. Should there be no domain-specific literature on human decision-making, practitioners may still follow a similar process as the one detailed below to capture human decision-making properties observable across domains, such as imperfect inter and intra-rater agreement^{32,33} and the existence of bias against protected attributes^{34–36}.

Internal consistency of human decision-making processes. When making a decision, experts follow an internal decision process based on the available information as well as their experience or training. However, even highly educated and trained individuals are still subject to flaws that are inherent to human decision-making processes, one of these being inconsistency. This is observed, for example, in a study involving seven experienced software professionals³², these were tasked with estimating the effort needed to complete sixty software development tasks over a period of three months. Six of the sixty tasks were estimated twice, in order to test the consistency of the experts. The authors find that the mean difference of the effort estimates of the same task by the same estimator was as much as 71%, with the correlation between the corresponding estimates being 0.7.

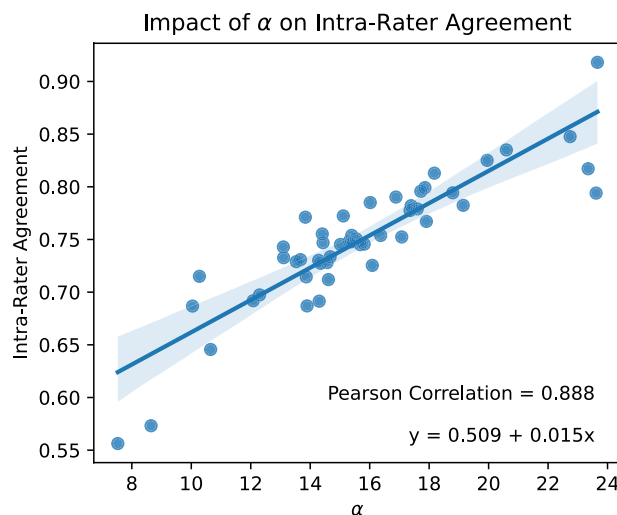


Fig. 5 Relationship between the parameter α and the intra-rater reliability of each expert. Although the value of the intra-rater reliability is also impacted by β_0 , β_1 , the correlation with α is apparent.

Evaluating the consistency of the decisions of a given expert, often referred to as “intra-rater consistency”, involves using a measure of the likelihood that a decision maker will produce a different decision when faced with the same inputs. In our framework, the probability that an expert will commit a mistake is calculated according to Equation (1). If an expert’s probabilities of error are consistently ≈ 0.5 , the resulting decision process would be entirely random. Sampling the expert decisions twice, with the same inputs, would lead to wildly different sets of predictions, meaning that the expert would have low intra-rater consistency.

To control the consistency of decision makers, we introduced the α parameter (see section [Controlling Variability and Expert’s consistency](#)), which controls how heavily the probability of error is swayed by the instance’s features. To measure the internal consistency of our synthetic experts over the entire dataset, we calculate their probabilities of error for each instance, and sample 10 sets of decisions for each expert, where the ground-truth is flipped with the calculated probability of error. We then calculate the value of Cohen’s κ , a common measure of agreement between two sets of predictions, for each possible pair of decisions, taking the average value to be our measure of the intra-rater agreement. If the probabilities of error are closer to the edges of the interval (0,1), then we expect the resulting sets of decisions to be much more similar between them. Note however, that the relationship between the intra-rater consistency and α is also impacted by different values of β_0 and β_1 . Should the values of β_0 , β_1 be 0 or 1, $\alpha = 0$ would also result in a deterministic process, as such, an increase within in the value of α will not always result in higher intra-rater reliability, particularly for lower values, but we expect to see an increase in intra-rater reliability as the value of α rises. In Fig. 5 we demonstrate that there is indeed a correlation between the value of α and the intra-rater reliability of a given expert.

In a high-stakes scenario, experts are highly trained, and therefore expected to exhibit substantial to perfect intra-rater agreement (Cohen’s κ of 0.6 to 1). Note, however, that the level of intra-rater agreement is not only a function of the expertise of the rater, but also of the difficulty of the classification task. In a study involving a series of cardiological diagnostic tests³³, intra-rater agreement of cardiologists was found to be consistently above $\kappa = 0.8$ for some medical tests. However, detecting some conditions was much harder even for experienced doctors, resulting in a lower intra-rater reliability $\kappa \in [0.4, 0.85]$. This was especially noticeable when the disease prevalence was very high or very low, as is the case in our financial fraud scenario. In Fig. 6 we show the distribution of intra-rater reliability throughout the expert team. Assuming our team of financial analysts is comprised of highly trained experts, performing a challenging task, this distribution seems realistic.

Testing L2D systems under variable intra-rater agreement can pose significant challenges, as these methods rely on modeling the expert behavior in order to predict their likelihood of being correct on a given instance. If a team is comprised of experts with low intra-rater consistency, they will have more unpredictable behaviour, harming the performance of L2D systems. In conclusion, having a team with variable levels of intra-rater consistency allows users to stress test L2D systems.

Inter-rater agreement. In high-stakes scenarios, experts have undergone extensive training to perform their decision-making task. Consequently, we expect the decision processes of different experts to be somewhat similar, according to commonly accepted practices or criteria in their domain of expertise. In other words, we expect to observe substantial levels of inter-rater agreement, a measure of the likelihood that two separate experts will produce the same prediction for the same input.

In the same study referenced in section [Internal Consistency of Human decision-making Processes](#), involving a team of cardiologists³³, the values of inter-rater agreement, as measured by Cohen’s κ , were as high as $\kappa = 0.97$. This value is, again, highly dependent on the classification task at hand, and its difficulty. It was found

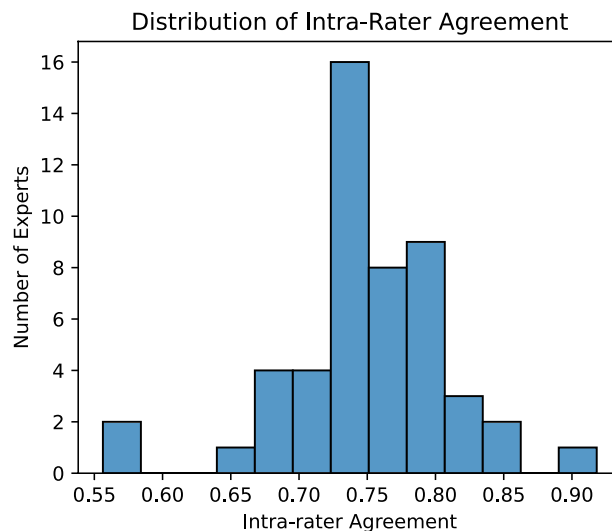


Fig. 6 Distribution of Intra-Rater agreement, calculated in months 4–8. The expert team contains various levels of intra-rater consistency.

that in a more challenging diagnostic task, inter-rater reliability was, on average, $\kappa = 0.49$. The proportion of agreement between experts was also reported at $p_a = 66.29\%$.

To determine the inter-rater agreement of our team of experts, we calculate Cohen's κ as well as the proportion of agreement between pairs of experts. In Fig. 7, we display a heatmap showing the value of κ for each expert pair. As the classification task is highly imbalanced, κ values are expected to be lower, as the expected level of agreement by chance rises.

We also report the values for the proportion of agreement, calculated as the fraction of instances where the predictions of a pair of experts are identical. These values, for each expert pair, shown in Fig. 8. In this set of results we can immediately notice how some of the experts within our team, such as experts #3 and #4, have a particularly unique set of decisions, yielding very low agreement with most of the team's members (an average of $\kappa = 0.33$ and $\kappa = 0.28$, respectively), but having high agreement between them ($\kappa = 0.71$).

Plotting the distribution for the two agreement metrics, in Fig. 9, we can see that the agreement between experts varies substantially, with $\kappa \in [0.06, 0.79]$, and $p_a \in [0.32, 0.95]$. We also note that the average values for the inter-rater agreement are similar to the real-world values reported in the previously mentioned study³³, with $\mathbb{E}[\kappa] = 0.49$ and $\mathbb{E}[p_a] = 0.75$.

When implementing L2D methods, it is desirable for inter-expert agreement not to be perfect. Should the experts perfectly agree on every instance, there would be no benefit in attempting to intelligently assign within that expert team. It is therefore beneficial for a L2D system that expert agreement be low, as long as the intra-rater consistency and the rater's performance is high, as this would result in a high number of instances where we can benefit from selecting the expert that is most likely to be correct. Our dataset allows users to extensively test L2D systems under high or low values of inter-rater agreement, by sampling experts in order to create teams with said properties.

Expert performance distribution. In real-world high-stakes scenarios, highly-trained experts within a field have been shown to have varying degrees of expertise. In a study²⁵ involving a panel of 8 US board-certified ophthalmologists with the highest values for intra-rater consistency, the authors find that their decision processes vary significantly as measured by their Recall and FPR. While we cannot derive a cost structure for the classification task, and can not, therefore, evaluate the difference in performance throughout the team, this study shows that the Recall and FPR of highly trained experts varies drastically throughout the team, with FPR varying between $\approx 1\%$ and $\approx 10\%$, while recall varies between $\approx 35\%$ and $\approx 99\%$.

In Equation (4), we establish a relationship between the expected misclassification cost for a given expert's decisions $\mathbb{E}[C]_j$ and their respective FPR_j and FNR_j . Let us consider the "Full Rejection" approach, where we reject all the alerts. If we substitute the expected cost of rejecting all the alerts into Equation (4), we obtain the set of FPR and FNR values that result in the same cost in the test split. As such, in a plot where the x axis is the FNR and the y axis is the FPR , all possible combinations of FPR and FNR leading to the same misclassification cost as rejecting all the alerts are represented by a single line. We exemplify this in panel'a' of Fig. 10, where the green area represents the combinations of FPR and FNR corresponding to a lower expected misclassification cost, while the red area represents a higher misclassification cost. In this figure, the blue dots represent each expert's performance metrics.

Every expert's performance, as measured by the expected misclassification cost, is assumed to be better than rejecting all alerts. We make this assumption because, in some current applications of alert review systems, deferral is done randomly to humans based on their availability¹. In these cases, if the experts do not perform, on average, better than just rejecting every single alert, they would not be useful for the system as a whole, and

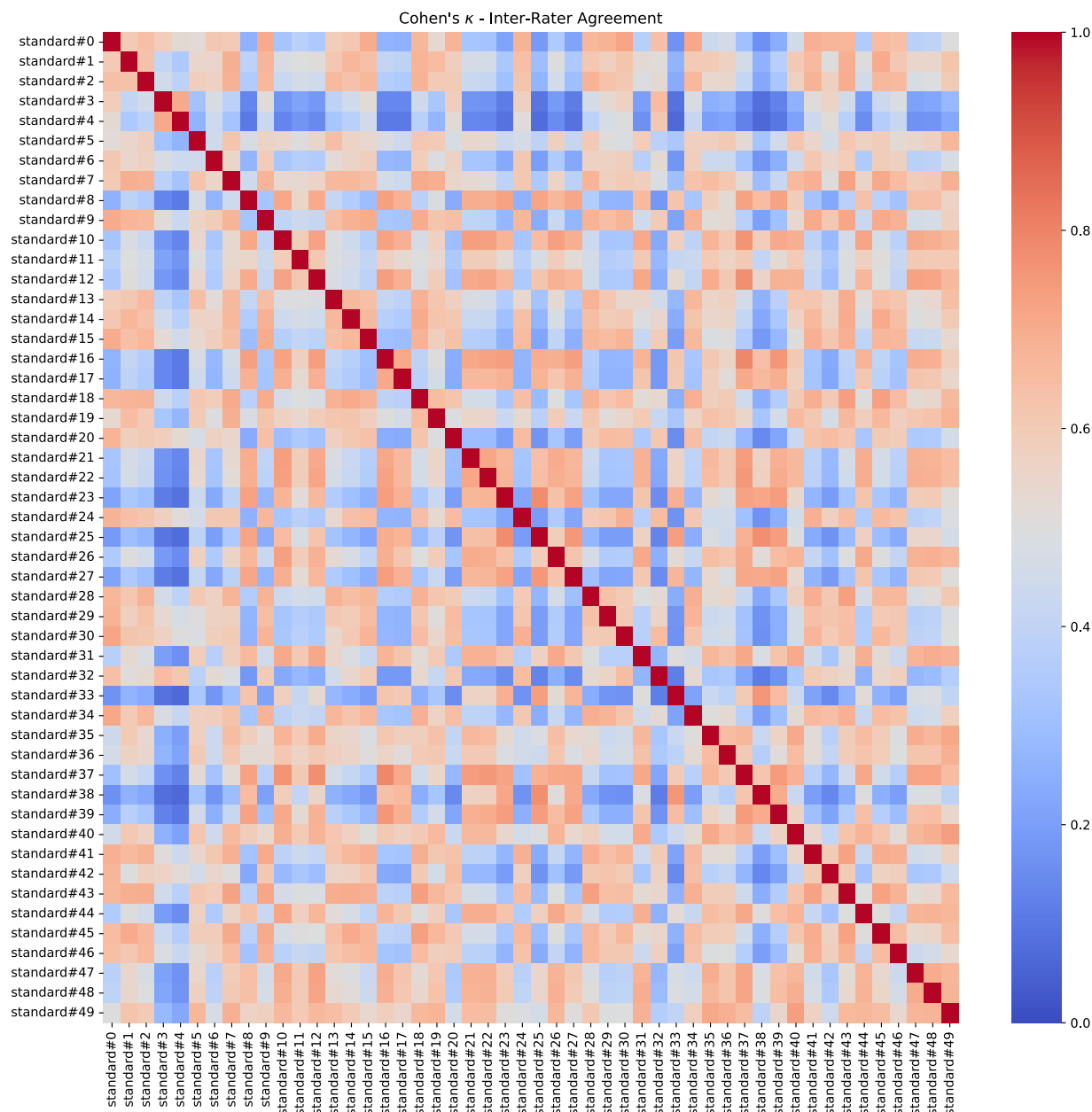


Fig. 7 Heatmap depicting the value of Cohen's κ for each expert pair. These values were obtained using the expert decisions in months 4–8, made available in our dataset.

there would be no reason to integrate them in the system. As such, to make our setup more realistic, we assume that our experts perform better than simply rejecting all the alerted applications. Panel 'b' of Fig. 10 shows the expected misclassification cost distribution calculated over months 4–8 in the dataset. Our dataset allows users to test L2D systems under a variety of team performances: by sampling only the highest or lowest performing experts, for example, one could test the impact that the average human performance has on the performance gains of such systems.

Unfairness against social groups. It is well-known that even highly-trained human experts can be unfair. A recent meta-analysis of hiring practices³⁴ shows that discrimination against candidates with disabilities, older candidates, less physically attractive candidates, and candidates with salient racial or ethnic traits is still an issue in Europe and the United States. The bias of human decision-makers is often influenced by the relationship between the social group and the ground-truth label in a given classification task, which is impacted by underlying social structures leading to differences across social groups³⁵. Let us consider the case of financial loan applications, where we assume a ground truth label of $Y = 1$ is the undesirable outcome, signifying that the applicant is not able to pay off the loan (i.e., rejected application). In the lending setting, even if the sampling process of loan applicants' income is unbiased and accurate, the data could still suffer from societal bias if some

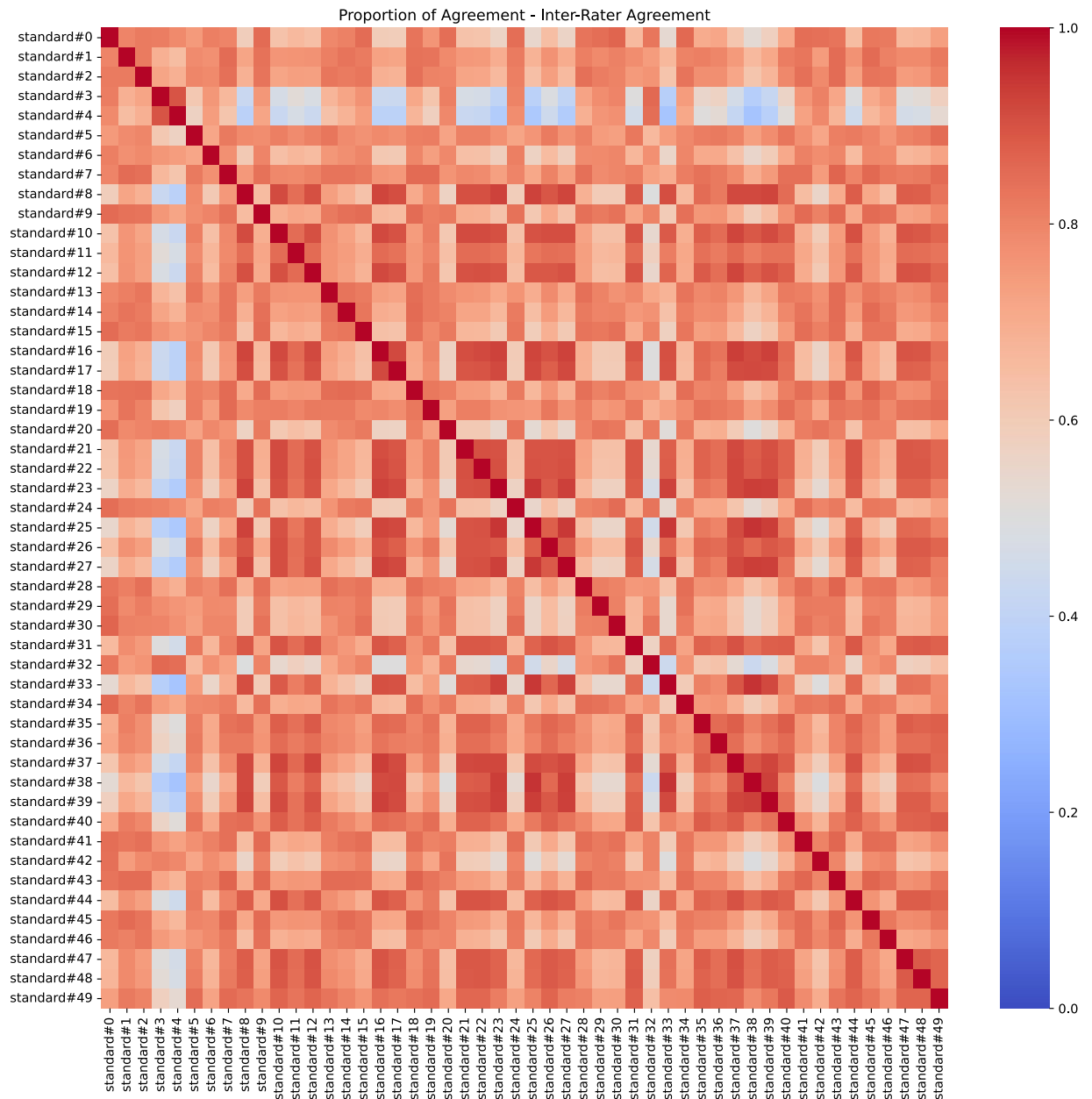


Fig. 8 Heatmap depicting the proportion of agreement p_d for each expert pair. These values were obtained using the expert decisions in months 4–8, made available in our dataset.

social group, for example women, systematically receive lower wages, making women less likely to be able to pay back a loan. In this case, a smaller fraction of women would have a positive ground truth label, meaning that a perfectly accurate algorithm would reject a higher proportion of women's applications when compared to men's applications, due to underlying socio-economic factor.

As such, to measure fairness, we need to take the prevalence of each group into account. To do so, we will use a predictive equality (pe) metric,

$$pe = \frac{\mathbb{P}(\hat{Y} = 1/Y = 0, G = 0)}{\mathbb{P}(\hat{Y} = 1/Y = 0, G = 1)}, \quad (9)$$

which, for a fair decision maker, should be $pe = 1$. Enforcing $pe \approx 1$ is often referred to as *predictive equality*³⁶. This requirement ensures that people from both privileged and underprivileged groups with a negative label are equally likely to be incorrectly classified. In the case of loan applications, this requirement ensures that people who are able to pay off the loan are granted said loan with the same likelihood independently of the social group they belong to. Going back to the hiring discrimination example, if candidates with similar qualifications are more likely to be accepted/rejected by virtue of their social group, it can be concluded that $pe \neq 1$, as the outcome ($\hat{Y}$) given the qualifications of the worker (Y) are influenced by their social group.

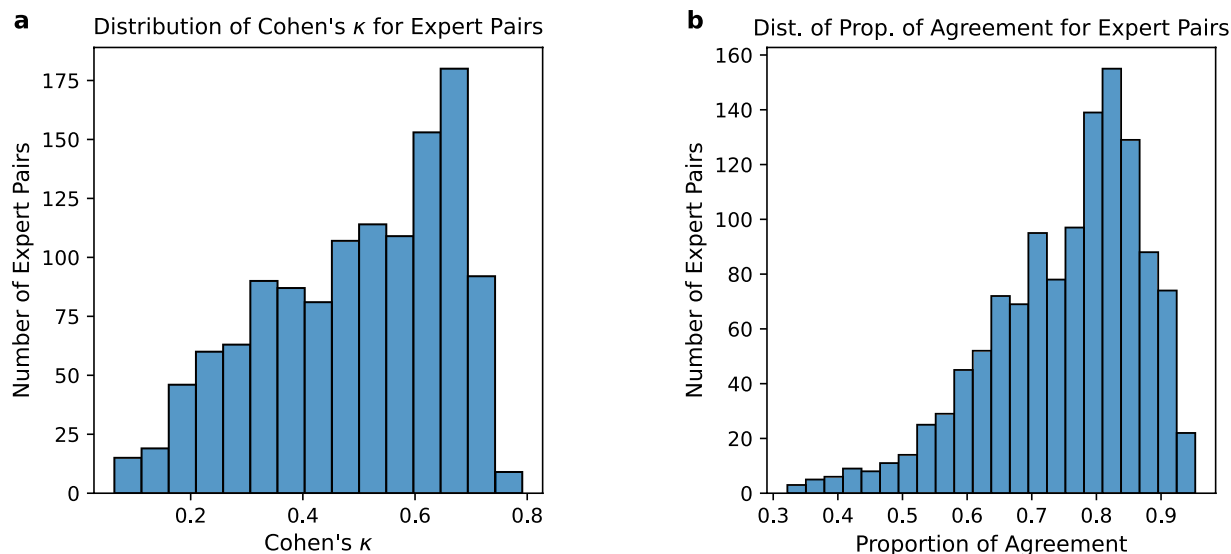


Fig. 9 (a) Distribution of Inter-Rater agreement as measured by Cohen's κ , calculated in months 4–8. (b) Distribution of Inter-Rater agreement as measured by proportion of agreement, calculated in months 4–8.

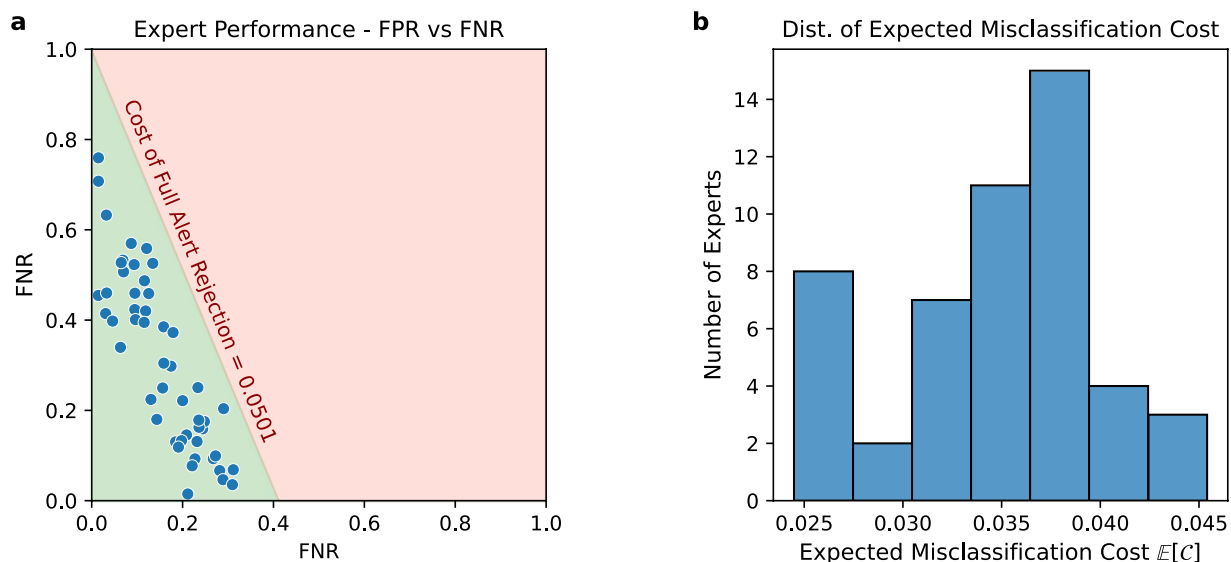


Fig. 10 (a) Expert Performance - FPR vs. FNR - Red line represents combinations of (FPR, FNR) resulting in the same cost as rejecting all customer applications. (b) Distribution of Expected Misclassification Cost in months 4–8. FiFAR's expert team contains a wide array of expertise, allowing for testing of L2D systems with varying levels of human performance.

In our dataset, two social groups are defined as “Older” customers (≥ 50 years old) and “Younger” customers (< 50 years old). The fraud prevalence within the alerts for the Older customers is $\approx 13\%$, while the prevalence for Younger customers is $\approx 11\%$. As such, we assume that experts can have varying levels of bias, but that most experts would exhibit bias against the older customers. In this case, we wish to ensure that no social group is being incorrectly denied a bank account at a higher rate than other social groups. In Fig. 11, we demonstrate the distribution of each of the expert's predictions p_e , as well as the group that they are biased against (the group with the highest false positive rate).

Our expert team is consistently biased against older customers, at varying degrees, with some experts being almost 50% more likely to incorrectly reject an older customer's application. Therefore, our dataset can be used to explore the impact that L2D systems have in the overall fairness of the produced decisions. Furthermore, it may also be used to develop fairness-aware methods, as L2D systems can have an active role in intelligently assigning instances in a way that promotes fairness (i.e., deferring the older customer's applications to the unbiased experts). While this is a concern that goes unaddressed by most recent publications in this area, it was

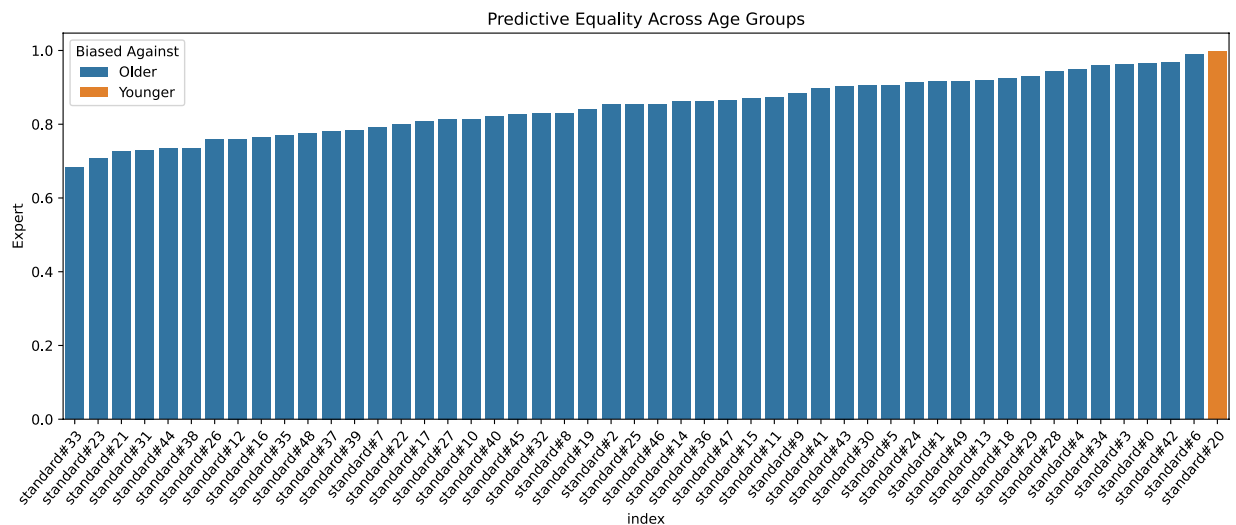


Fig. 11 Expert Predictive Equality - Experts who are biased against a given group have a higher false positive rate (incorrectly rejecting an application) within said group.

explored by the original authors in their paper¹⁰, where they consider adding a fairness regularization term to the loss function of their L2D system.

Feature dependence and algorithmic bias. This section focuses on validating whether our experts' feature dependence and algorithmic bias (dependence on the Alert Model's score) are realistic. To conduct this experiment, we will use a set of real-world financial analyst decisions. This dataset is private and, therefore, can not be made available in this publication. These analysts' decisions are made based on a set of features that are shown to them, including the score of an ML model trained to predict fraud. This dataset only contains one expert's prediction per instance, and, therefore, could not be used to evaluate the properties of our synthetic analysts in the past sections. Furthermore, as this dataset contains only the decisions made by the experts, and not the ground-truth label, we cannot analyse the dependency between the features and the probability of error, or the distribution of experts performance. We will use this dataset to evaluate the dependency between the decisions of each expert and the features presented to them.

To establish a level of dependency of an expert's decisions on a given feature, we fit a *LGBMClassifier* to the decisions of each expert, and measure each feature's importance within the resulting model. The relative importance of each feature is calculated as the proportion of information gain, in training, that can be attributed to splits based on said feature. We repeat this process with our synthetic experts' decisions, in order to compare the results. Each real expert model was trained by splitting the data pertaining to each expert into 60-20-20 splits. As the set of decisions pertaining to each of the 20 real-world experts had an average of 10 K instances, we also trained the models for our synthetic experts with 10 K predictions, following the same train-validation-test splitting strategy, ensuring that the training set was sampled from months 4–6, the validation set was sampled from month 7, and the test set was sampled from month 8. These models were trained by minimizing the binary cross-entropy. The hyperparameter selection process is detailed in Table 3.

Figure 12 displays the ROC curves in the test set for the models trained, showing that both sets of *LGBMClassifiers* were able to model the human and synthetic expert behavior. However, the values for the area under the curve (AUC) are much lower for the real-world experts. In Fig. 13, we observe that the maximum value of AUC observed for the real experts is lower than the lowest AUC observed for the synthetic experts. The source of this disparity in the predictability of the expert behaviours may be due to several factors. It may be the case that the real-world financial analysts have a lower intra-rater consistency than the generated experts, making their decision process inherently less predictable. However, it is also possible that the decision process of the real-world experts is significantly more complex than our simulated experts, making it harder to model with limited data. As we have no access to the ground truth labels, and each instance only has the prediction of a single expert, we are not able to determine the performance and intra-rater reliability of the real-world experts, making it impossible to identify the source of this disparity.

Panel'a' of Fig. 14 displays the distribution of feature importances for the *LGBMClassifiers* modeling the team of 20 real-world fraud analysts. The most important feature in predicting the decisions of the experts is the *Model Score*, which is to be expected as the value of the model score will not only influence the decision of financial analysts²¹, but is also strongly correlated to the probability of fraud. We can see that, although some features exhibit higher importance across all experts, the distribution of feature importances is relatively even, with some outliers, which can be interpreted as experts whose decisions are unusually dependent on said feature.

With our synthetic experts, we aimed to obtain a similar distribution, with a high dependence on the model score, a few features with higher feature importance, followed by a relatively even distribution of feature importance for the rest of the features. Our synthetic experts' feature dependence, as measured by the same method, is represented in panel'b' of Fig. 14. In these results, we can see that the value of the proportion Gains are, across

Hyperparameter	Values or Interval	Distribution
boosting_type	"goss"	
enable_bundle	False	
n_estimators	[50,300]	Log
max_depth	[2,20]	Unif.
num_leaves	[10,100]	Log
min_child_samples	[5,500]	Log
learning_rate	[0.01, 2]	Log
reg_alpha	[0.0001, 0.1]	Log
reg_lambda	[0.0001, 0.1]	Log

Table 3. Feature Dependence Testing: LightGBM hyperparameter search space. At each trial, hyperparameter sampling is conducted via the Tree-structure Parzen Estimator (TPE) algorithm, as implemented in the Optuna⁴¹ package. The optimization objective is to maximize the area under the ROC curve in validation. Optimization is run for 50 rounds with early-stopping after 10 rounds without improvement in the optimization objective. The total time for hyperparameter optimization for the real expert models was 4 minutes using 30 cores of an Intel(R) Xeon(R) Gold 5120 CPU @ 2.20 GHz. For the synthetic expert models the total time was 8 minutes using 10 cores.

the board, higher for all features. This is explained by the fact that the BAF dataset contains 30 features, while the dataset of real-world financial analyst decisions contains 116, which means that the feature importance will be spread out over a much larger number of features. In both expert teams, the model score is the most important feature, followed by a similar distribution of feature importances. This experiment validates that our synthetic financial analysts behave in a similar feature dependence to the real-world experts, even though the decision process of a real analyst may be more complex. A more detailed visualization of the LGBM feature importances for the real and synthetic expert teams is featured in Figs. 15 and 16, respectively.

L2D benchmarking. In this section, we will demonstrate how FiFAR can be used for robust testing of L2D methods, under realistic conditions. We will only consider multi-expert methods that have been adapted to assign under human work capacity constraints. To the best of our knowledge, only one paper³⁷ considers the multi-expert learning to defer setting under individually defined work capacity constraints.

We will use the same baselines as said study (Full Rejection, Only Classifier, Random Assignment, OvA), as well as their newly proposed method (DeCCaF):

- Full Rejection: all alerts are automatically rejected.
- Only Classifier: all final decisions on the alerts are made by the classifier h , trained according to the L2D algorithm to predict alongside the experts.
- Random Assignment: alert deferral is done randomly throughout the experts/model until each of their capacity constraints are met.
- OvA: this method involves training one classifier per expert, which predicts the likelihood that an expert will make the correct prediction on said instance. Each of these are trained on the subset of training data corresponding to the set of a given expert's decisions. A separate classifier h is trained to predict the probability that the instance belongs to the positive class. The classifier h is trained on all of the instances in the training set.

In testing, this method defers to an expert if they are predicted as the most likely to be correct, or to classifier h if the probability of the instance being fraudulent/legitimate is higher than that of any expert being correct. Deferral is done instance-wise to the expert/model that is most likely to be correct, until their work capacity is exhausted, at which point it chooses to defer to the second most likely to be correct, and so forth.

- DeCCaF: this method trains a classifier for the entire team, by also taking as input the *expert_id* of the expert which made the prediction for a given instance in training. This classifier then outputs the probability that an expert will make the correct prediction on a given instance. This method also trains a separate classifier h , identically to the OvA method.

In testing, this method maximizes the global probability of correctness by using constraint programming.

Training classifier h , OvA and DeCCaF. Of the aforementioned baselines, only OvA and DeCCaF can strictly be considered L2D methods, as they estimate human performance to optimize deferral. Both OvA and DeCCaF use a separate ML classifier as the automated decision-maker in their algorithm. Consequently, to reduce the differences between the baselines, the "Only Classifier", "OvA" and "DeCCaF" approaches will share the same classifier h . The "Full Rejection" and "Random Assignment" approaches do not require the training of any additional models.

Classifier h . We implement h with a LightGBM¹⁹ classifier, trained on the alerts raised by the Alert Model, ranging from months 4–6 of the BAF dataset. The model is trained by minimizing the weighted log-loss, where the

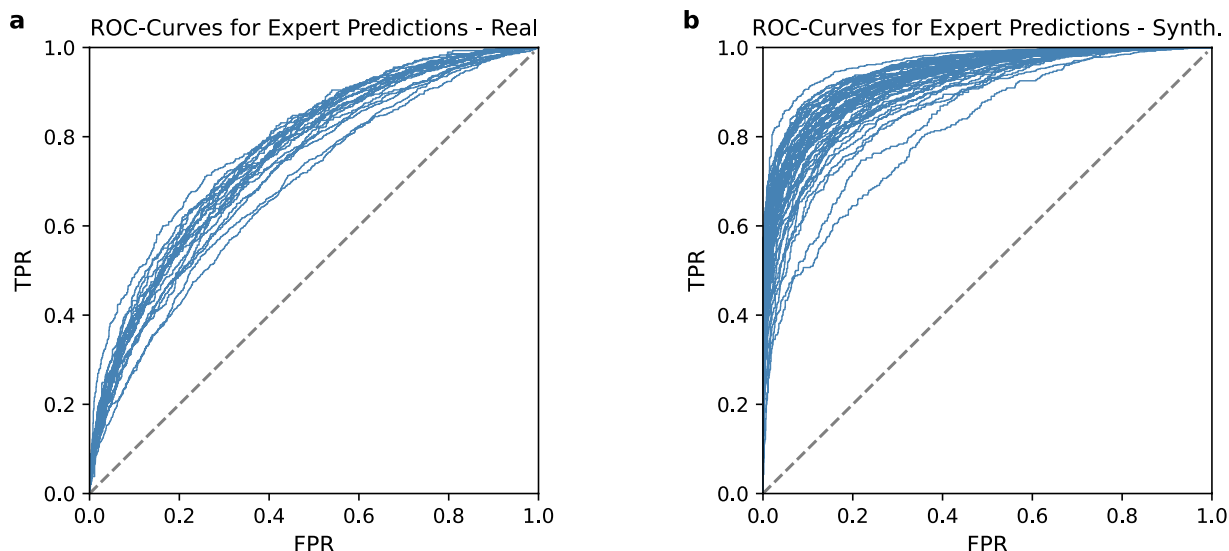


Fig. 12 (a) ROC Curves in Test Split - Real Expert (b) ROC Curves in Test Split - Synthetic Experts.

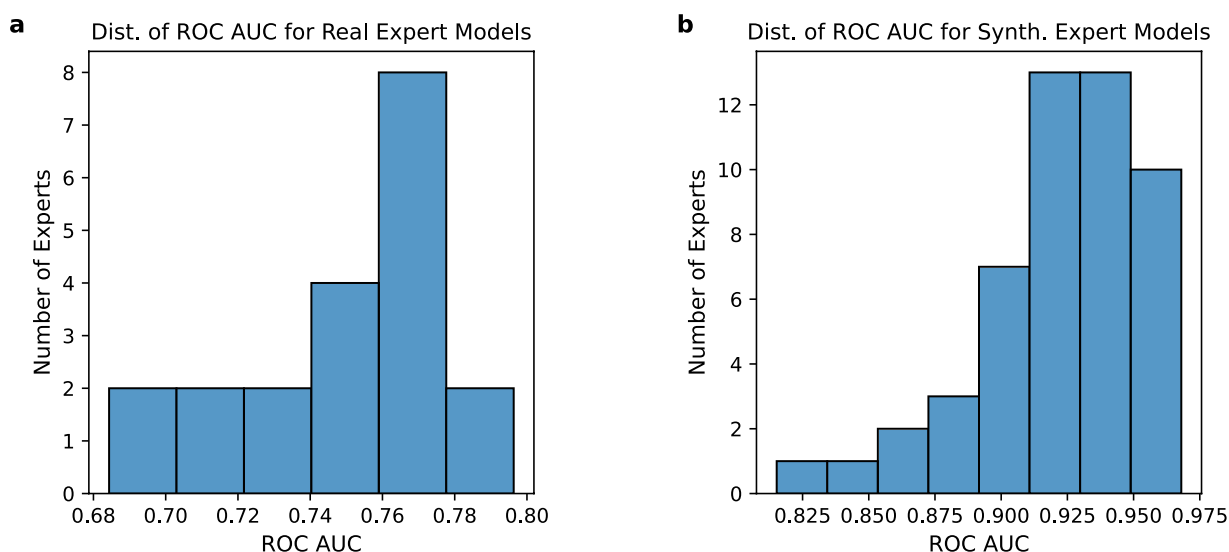


Fig. 13 (a) ROC-AUC Values in Test Split - Real Experts (b) ROC-AUC Values in Test Split - Synthetic Experts.

weight of a label positive instance is set to $c_i = 1$, and the weight of a label negative instance to $c_i = \lambda$. The process for hyperparameter selection is detailed in Table 4. Note that, in training this model, we also test several values for the initial probability estimate of the base predictor of the boosting model. This was done in order to test if introducing a bias towards predicting fraud can be beneficial to our model, as across all scenarios, false negatives incur a higher cost. As LightGBM classifiers are gradient boosting models, they must start from an initial estimator, which in this case makes a constant prediction for the probability of an instance being fraudulent. Given instance-wise feature weights c_i , the default value $g_{initial}$ for the logit of the initial estimator's prediction is

$$g_d = \text{logit} \left(\frac{\sum_i c_i y_i}{\sum_i c_i} \right) = 0.782, \quad (10)$$

which is the value that minimizes the weighted log-loss assuming that the model outputs g_d for every instance in training. When training the model, we run the hyper-parameter search independently for $g_{initial} \in \{g_d, g_d + 0.2, g_d + 0.4, \dots, g_d + 2\}$, the upper limit was selected as performance starts drastically deteriorating past said value. The values for the selected model are displayed in Table 4.

OvA Classifiers and DeCCaF Team Model. We model both the OvA classifiers and the DeCCaF team model with LightGBM¹⁹ classifiers. These models are trained on the alerts raised by the Alert Model, and the

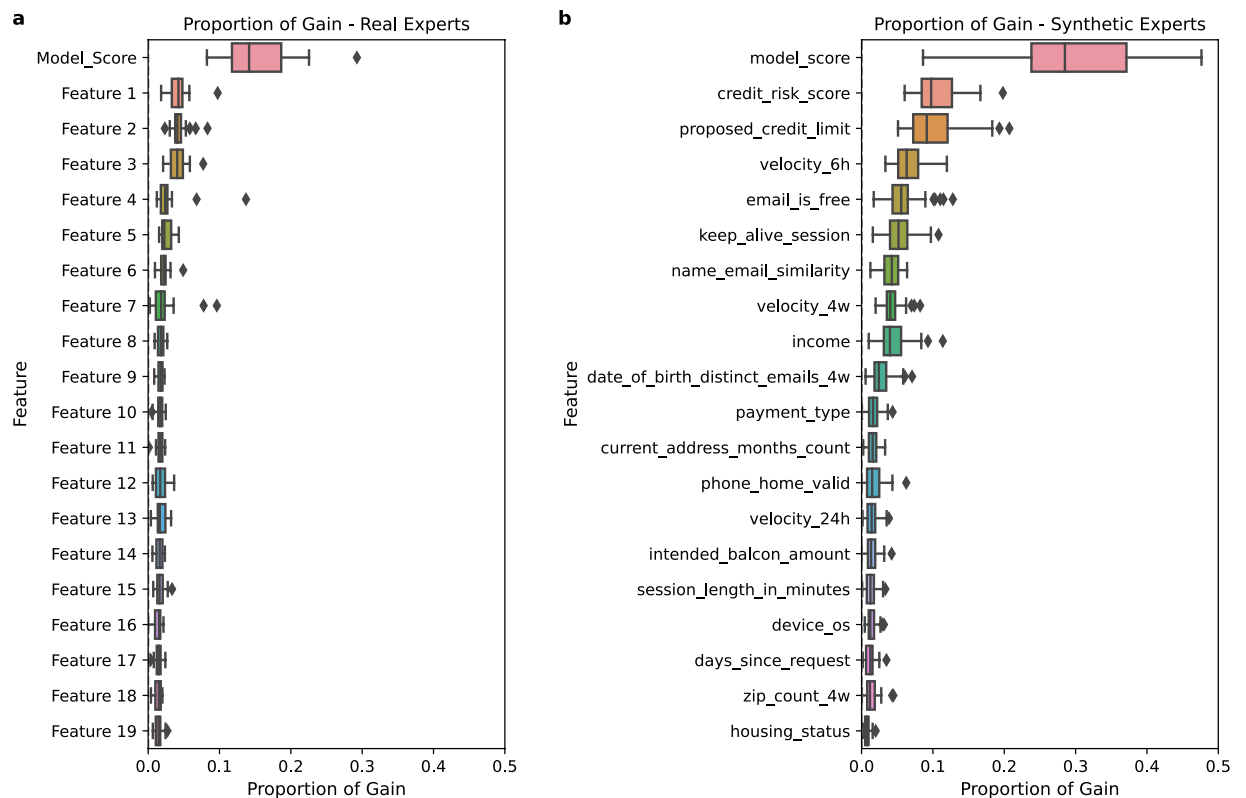


Fig. 14 (a) Distribution of Feature Importances - Real Experts. Features are enumerated rather than named due to privacy related concerns regarding the real analyst decision data (b) Distribution of Feature Importances - Synthetic Experts.

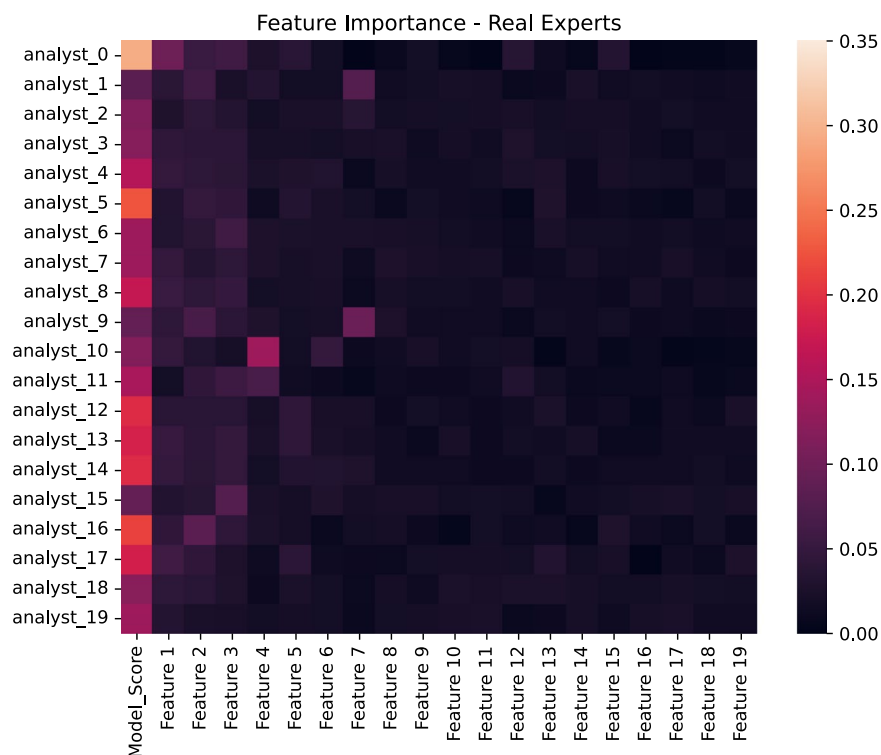


Fig. 15 Heatmap of Feature Importances - Real Experts. Features are enumerated rather than named due to privacy related concerns regarding the real analyst decision data.

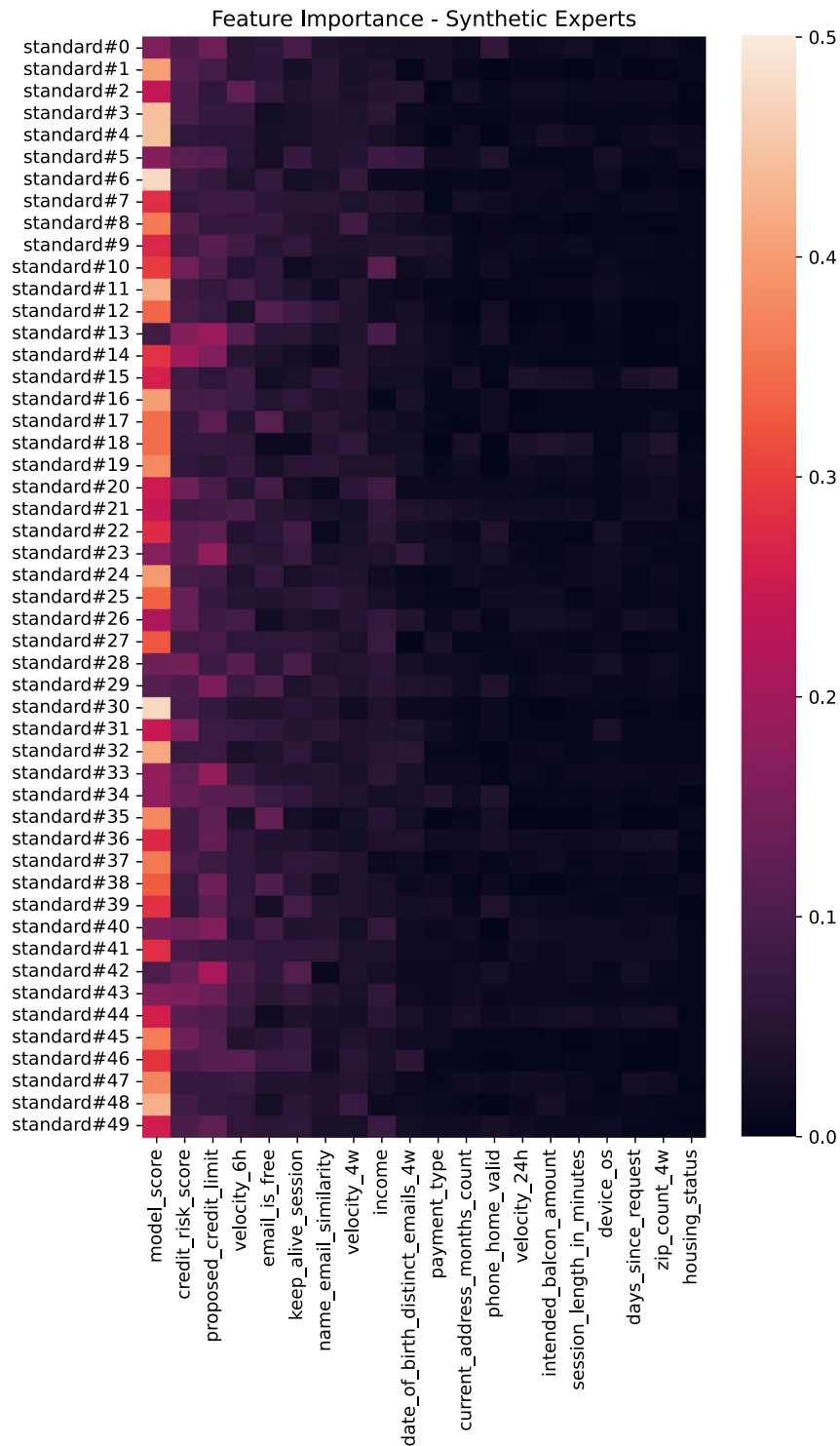


Fig. 16 Heatmap of Feature Importances - Synthetic Experts.

corresponding expert predictions, ranging from months four to six of the BAF dataset. Both methods are trained by minimizing the weighted log-loss. The hyperparameter selection process is detailed in Table 5. The OvA classifiers and the DeCCaF team model were trained in the 25 distinct training scenarios.

Predictive performance. In this subsection we evaluate the ability of the aforementioned models to perform their classification tasks. In the case of the classifier h , this involves testing its ability to predict fraud, while, in the case of the OvA Classifiers and the DeCCaF team model, this involves testing their ability to model the probability of each expert's correctness.

Hyperparameter	Values or interval	Distribution	Selected
init_score	{0.782, 0.982, ..., 2.582}		1.982
boosting_type	"dart"		"dart"
enable_bundle	[False, True]		False
n_estimators	[50, 250]	Unif.	160
max_depth	[2, 5]	Unif.	2
num_leaves	[100, 1000]	Unif.	708
min_child_samples	[5, 200]	Unif.	89
learning_rate	[0.001, 1]	Unif.	0.779
reg_alpha	[0.001, 2]	Unif.	1.673
reg_lambda	[0.001, 2]	Unif.	1.932

Table 4. Classifier h: Initial LightGBM hyperparameter search space. At each trial, hyperparameter sampling is conducted via the Tree-structure Parzen Estimator (TPE) algorithm, as implemented in the Optuna⁴¹ package. The optimization objective is to minimize the weighted log-loss in validation. Optimization is run separately for each possible value of "init_score", for a total of 300 rounds with 200 startup rounds. The total time for hyperparameter optimization was 20 minutes using 10 cores of an Intel(R) Xeon(R) Gold 5120 CPU @ 2.20 GHz. As models with max_depth = 2 consistently outperformed others, the optimization was rerun for 1700 trials with 1500 startup rounds, with the same objective, but max_depth fixed at 2, in order to fine-tune the model as best as possible. The displayed selected values result from this second search, which took a total of 90 minutes using the same computational resources.

Hyperparameter	Values or Interval	Distribution
boosting_type	"dart"	
enable_bundle	[False, True]	
n_estimators	[50, 250]	Unif.
max_depth	[2, 20]	Unif.
num_leaves	[100, 1000]	Log.
min_child_samples	[5, 100]	Log.
learning_rate	[0.005, 0.5]	Log.
reg_alpha	[0.0001, 0.1]	Log.
reg_lambda	[0.0001, 0.1]	Log.

Table 5. LightGBM hyperparameter search space - OvA Classifiers and DeCCaF team model. At each trial, hyperparameter sampling is conducted via the Tree-structure Parzen Estimator (TPE) algorithm, as implemented in the Optuna⁴¹ package. The optimization objective is to minimize the weighted log-loss in validation. Optimization is run for a total of 120 rounds with 100 startup rounds. The total time for hyperparameter optimization, in all 25 training scenarios, using 10 cores of an Intel(R) Xeon(R) Gold 5120 CPU @ 2.20 GHz, was 4 hours and 12 minutes for the OvA classifiers, and 1 hour and 23 minutes for the DeCCaF team models.

Classifier h. To evaluate the ability of this model to rank the instances according to their probability of belonging to the positive class, we use the ROC AUC, which has a value of 0.71, indicating that the classifier h is able to correctly rank instances according to the probability of fraud.

To evaluate the accuracy of the probability estimates, we use the *expected calibration error*³⁸ (ECE), a measure of the mean absolute deviation of the probability estimates to the real probabilities of error; the ECE obtained in testing was 4.9%.

OvA Classifiers and DeCCaF team model. The mean values of the OvA classifiers' AUC and ECE, with the 95% confidence intervals, are reported in Table 6. In the paper where the OvA multi-expert method is introduced¹², the authors also report the ECE in three separate datasets, with values rarely surpassing 4%. In this task, however, the miscalibration of the predictions is significantly higher, rarely falling below 5%. This may be due to several properties of our dataset, as the decision process of our simulated experts is more complex, the dataset contains concept drift and shifts in prevalence throughout the months, and the high class imbalance pose significant challenges.

The mean values of the DeCCaF team model's AUC and ECE, with the 95% confidence intervals, are also reported in Table 6. It seems that in this scenario, jointly modeling the team's behavior leads to a slight increase in the average AUC (4% on average), and a significant improvement in the calibration of the estimates (17% on average). Note that for both methods, the values of the AUC for the prediction of expert correctness do not surpass 0.65, showing that under these realistic training conditions, it may be challenging to model human behaviour.

team	auc_ova	ece_ova	auc_deccaf	ece_deccaf
team_1	0.61 ± 0.01	6.0 ± 0.5	0.64 ± 0.01	4.6 ± 0.5
team_2	0.58 ± 0.01	6.0 ± 0.6	0.6 ± 0.01	4.8 ± 0.4
team_3	0.6 ± 0.02	5.5 ± 0.5	0.62 ± 0.02	5.2 ± 0.6
team_4	0.59 ± 0.01	5.4 ± 0.6	0.6 ± 0.01	3.8 ± 0.4
team_5	0.59 ± 0.01	5.5 ± 0.7	0.61 ± 0.01	4.9 ± 0.6

Table 6. OvA Classifier and DeCCaF Team model - ROC AUC and Expected Calibration Error (ECE).

Team	Full Rejection	Only Classifier h	Random Deferral	OvA	DeCCaF
team_1	213.5	204	169.6 ± 2.53	151.7 ± 1.99	138.1 ± 5.14
team_2	213.5	204	157.8 ± 1.39	142.0 ± 1.61	145.3 ± 4.28
team_3	213.5	204	151.2 ± 1.84	131.3 ± 2.19	126.2 ± 4.52
team_4	213.5	204	163.1 ± 1.61	145.8 ± 3.51	141.6 ± 5.32
team_5	213.5	204	163.3 ± 1.58	141.2 ± 2.57	132.0 ± 1.93

Table 7. Deferral Results.

Deferral Results. Having determined that the models are able to model both the probability of fraud and the human probability of error, we conduct an evaluation of the quality of deferral of each system. The quality of deferral will be measured by the average misclassification cost incurred over the entire test set. The results are displayed in Table 7 with a 95% confidence interval.

As expected, the worst option is Full Rejection, as all possible decision makers (experts and classifier h), have a lower misclassification cost over the test set. This is evidenced by the “Only Classifier h” column, as well as panel a’ of Fig. 10. Experts have, on average, a higher performance than the Classifier h, and, therefore the “Random Assignment” baseline is the best non-L2D baseline.

Finally we observe that both OvA and DeCCaF obtain significantly better results than all non-L2D baselines, with the methods achieving an average 11% and 15% decrease, respectively, in the misclassification cost over the Random Assignment baseline. Note that while DeCCaF significantly outperforms the OvA approach for “team_1” and “team_5”, there is overlap between the error bars within the rest of the scenarios.

These results demonstrate the importance of testing L2D systems with complex expert behavior under a wide variety of scenarios, as even the relative ranking of these algorithms can be impacted by changes in the composition of the team.

Limitations of OpenL2D and FiFAR. A key restriction of our framework is that the creation of realistic synthetic experts requires an understanding of human decision-making literature. Over the course of this paper, we aimed to explain the reasoning behind each parameter involved in the data generation process, as well as its relationship to certain properties of human decision-making teams such as intra and inter-rater agreement and biases against protected groups. Should an OpenL2D user find there to be a lack of literature regarding their specific use case or domain, they can still apply the framework to their dataset, as long as human decision-making traits observed across a wide variety of domains are taken into account (*i.e.*, human experts often display biases, inter-rater, and intra-rater agreement should not be perfect, human decisions tend to be more dependent on features that correlate highly with the label, etc.)

Another critical limitation of our proposed data generation framework is its exclusive focus on binary classification problems. The choice to focus its design on these scenarios comes down to a few factors. First, the binary classification setting allows us to establish an intuitive relationship between the probabilities of false positive and false negative mistakes, as stated in the Section **Synthetic Expert Generation**: “The weights are shared in both formulas, with opposite signs, ensuring that if a feature contributes to increase the FP probability, it will, in turn, decrease the FN probability. We choose to enforce this restriction in order to produce a more intuitive decision-making process: if an expert does more FPs in a given region of the feature space, we also expect them to do fewer FNs, assuming they will be biased towards classifying more instances as positive in said region.”. However, the same logic cannot be directly applied to the multiclass setting, as failing to correctly classify an instance does not necessarily imply the selection of a specific class. While one could easily apply our label noise approach to a multiclass problem, by, for example, generating a single probability of error for each instance, and sampling a random incorrect class with said probability, this would not be realistic. In the commonly used CIFAR-10¹⁶ dataset, it is fair to assume that a human annotator, upon observing an image belonging to the “truck” category, is more likely to misclassify said image as an “automobile” rather than a “dog”. As such, realistic simulation of human error on a multi-class classification task is significantly more complex than the binary scenario, as, for each possible incorrect class e , this would involve the generation of an adequate $\mathbb{P}(m_{i,j} = e | y_i = c, \mathbf{x}_i)$, $e \in \mathcal{Y} \setminus \{c\}$, where $\mathcal{Y}$ is the label space. This would involve creating a specific set of weights for each possible error type in the confusion matrix, which would be highly dataset specific and thus unfeasible in a framework that aims to be as dataset agnostic as possible. Second, many high-stakes decision making problems involve binary classification, such as financial fraud detection^{19,20}, medical screening²⁵ and

credit risk scoring³⁹, meaning our framework can be applied to a wide variety of domains in spite of its incompatibility with multi-class classification.

Our framework is also limited to tabular data. While application of this framework to image or text datasets could, in practice, be achieved by converting said data to vector embeddings, we discourage this approach altogether, as responsible and realistic use of our framework requires an understanding of the meaning of each feature, while vector embeddings are, by their very nature, uninterpretable.

Due to the use of a feature-dependent label-noise approach^{17,18}, our framework is not applicable to decision-making tasks where no ground truth labels are present. However, we consider the generation of synthetic experts in these scenarios outside of the scope of this work, as current research on L2D hinges on the existence of ground-truth labels, as well as human decisions. While there is work considering the existence of scarce human labels, tackling such issues by using either meta-learning or active learning^{13,40}, the assumption that ground-truth labels exist is common to all cited works, and is considered necessary for an L2D algorithm to be able to learn to estimate the human's probability of correctness for a given instance.

We also acknowledge that the process of tweaking the parameters of our framework until the desired behavior is achieved is time-consuming, as the interaction between each variable is not trivial. While a real-time, interactive visualization of the impact of each change on the expert team (i.e., by controlling each parameter with sliders and viewing a change's impact on key metrics such as inter and intra rater agreement) would be ideal, this is impossible due to computational restrictions. The process of generating a team of experts, ensuring the desired performance via the methods described in [Controlling Expert Performance](#), as well as calculating statistics such as performance and inter-rater agreement in a significant portion of the dataset is far too time-consuming to allow for a real-time display of a simulation's results. However, to address this issue in the best manner possible, we have created a notebook which can be used for analysis of the team's properties, displaying statistics on inter and intra-rater agreement, performance, bias against each protected attribute and dependence on each feature (see Section Code availability). This notebook is completely dataset and generation agnostic, meaning that independently of the dataset used and the generation parameters, users can run this notebook after each change in the framework's parameters to observe key statistics regarding their simulated experts' decision-making properties, displaying statistics on inter and intra-rater agreement, performance, bias against each protected attribute and dependence on each feature.

Regarding FiFAR as a ready-to-use dataset, uncritical use of FiFAR (i.e. without conducting proper robustness analysis in changes to deferral rate, expert availability and workload distribution) could yield unreliable reporting of an L2D system's performance. Using our experimental setting as an example, (see section [L2D Benchmarking](#)), should a user only conduct testing with the experts present in "team_1" (see [Deferral Results](#)), they could erroneously conclude that DeCCaF is a clear improvement over the OvA method. In our experiments, however, we show that the relative performance of these two algorithms is significantly impacted by the choice of experts, implying that such definitive claims can not be made about the relative performance of these algorithms. We encourage practitioners using our framework to conduct similarly rigorous testing by fully using the resources made available in FiFAR, including the extensive training and testing scenarios.

Finally, we wish to emphasize that our synthetic experts can not replace, in any way, humans involved in HAIC systems, as it would be necessary to gather real-world expert data in order to train the system to be used in a real-world application. OpenL2D and FiFAR are meant to be used as a benchmarking tool only, allowing for the study of new L2D approaches on better synthetic data than currently used. It could be argued, however, that by using these synthetic experts for research purposes, we are affecting the livelihood of large scale annotation workers (i.e. MKTurk), which researchers often resort to. However, in some use cases, where domain-specific expertise is needed, such annotation services may not be of use to practitioners, and it may be impossible/unfeasible to obtain real human expert data. In cases where human predictions are accessible and pertinent to the use case, researchers should prefer these over synthetic expert predictions, as these would constitute real human behavior. For these reasons we believe that our framework motivates the use of more complex synthetic expert data only when real human predictions are unattainable, without posing a threat to current existing annotation services.

Usage Notes

The FiFAR dataset is publicly accessible at <https://doi.org/10.6084/m9.figshare.28351172>. In order to ensure reproducibility, the Alert Model, as well as all models used in the L2D benchmark are also made available. The FiFAR dataset can be reproducibly re-generated by running the code detailed in Section Code availability. Users are encouraged to modify the dataset and adapt it to their use case, making full use of the flexibility of our framework. Evaluation notebooks are included in order to validate the quality of the generated synthetic experts. These contain the code used to conduct the experiments presented in the [Technical Validation](#) Section, which will be detailed in the Code availability Section.

Code availability

The OpenL2D synthetic data generation framework will be made available for public use in GitHub (<https://github.com/feedzai/openl2d/tree/main>). In this repository, we also include the code needed to train the Alert Model used, and to perform all the data processing needed for the generation of the FiFAR dataset. The repository also contains the model object for the Alert Model, allowing for reproducible generation of the synthetic expert predictions.

Furthermore, we also include the results of the benchmark conducted in Section [L2D Benchmarking](#) in the repository, allowing users to thoroughly scrutinize the deferral methods included and to ensure reproducibility. In this way, our work can function as a constantly expanding benchmark by simply adding new methods and their results to the existing resources.

To ensure reproducibility of every step in the data generation process, we also give users access to the python environment specification file. Detailed instructions on how to use the framework are available in the GitHub repository, in the “README.md” file.

Our repository also includes a folder (“OpenL2D_Use_Example”) complete with several resources to facilitate the use of our framework. For this example, we use the popular “Adult” dataset, aimed at predicting whether an individual’s income is greater than 50 K dollars per year, based on demographic information. The folder contains a subdirectory named “data”, where the relevant files of the “Adult” dataset are contained, alongside a notebook detailing the preprocessing necessary to prepare a dataset for the OpenL2D framework. An example expert configuration file is also included so that users can easily start experimenting by tweaking different parameters. This folder also includes the “expert_analysis.ipynb” notebook, which is completely agnostic to the parameters of the generation, meaning it can be used to observe the key statistics of the generated expert team. This notebook serves the purpose of facilitating the iterative process of tweaking the generation parameters until the desired behavior is achieved.

Received: 13 June 2024; Accepted: 18 February 2025;

Published online: 23 April 2025

References

- De-Arteaga, M., Fogliato, R. & Chouldechova, A. A Case for Humans-in-the-Loop: Decisions in the Presence of Erroneous Algorithmic Scores. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, 1–12 <https://doi.org/10.1145/3313831.3376638> (ACM, Honolulu HI USA, 2020).
- Dellermann, D., Ebel, P., Soellner, M. & Leimeister, J. M. Hybrid. *Intelligence. Business & Information Systems Engineering* **61**, 637–643, <https://doi.org/10.1007/s12599-019-00595-2> (2019).
- Inkpen, K. *et al.* Advancing human-ai complementarity: The impact of user expertise and algorithmic tuning on joint decision making. *ACM Trans. Comput.-Hum. Interact.* **30**, <https://doi.org/10.1145/3534561> (2023).
- Charusaie, M.-A., *et al.* (eds.) *International Conference on Machine Learning, ICML 2022, 17–23 July 2022, Baltimore, Maryland, USA*, vol. 162 of *Proceedings of Machine Learning Research*, 2972–3005 <https://doi.org/10.48550/arXiv.2207.09584> (PMLR, 2022).
- Hemmer, P., Schellhammer, S., Vössing, M., Jakubik, J. & Satzger, G. Forming Effective Human-AI Teams: Building Machine Learning Models that Complement the Capabilities of Multiple Experts. In Raedt, L. D. (ed.) *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23–29 July 2022*, 2478–2484 <https://doi.org/10.24963/ijcai.2022/344> (ijcai.org, 2022).
- Raghu, M. *et al.* Direct uncertainty prediction for medical second opinions. In *International Conference on Machine Learning*, 5281–5290 <https://doi.org/10.48550/arXiv.1807.01771> (PMLR, 2019).
- Raghu, M. *et al.* The Algorithmic Automation Problem: Prediction, Triage, and Human Effort. *CoRR abs/1903.12220* <https://doi.org/10.48550/arXiv.1903.12220> (2019).
- Mozannar, H. & Sontag, D. A. Consistent Estimators for Learning to Defer to an Expert. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13–18 July 2020, Virtual Event*, vol. 119 of *Proceedings of Machine Learning Research*, 7076–7087 <https://doi.org/10.48550/arXiv.2006.01862> (PMLR, 2020).
- Mozannar, H. *et al.* Who should predict? exact algorithms for learning to defer to humans. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (AISTATS)*, <https://doi.org/10.48550/arXiv.2301.06197> (2023).
- Madras, D., Pitassi, T. & Zemel, R. Predict Responsibly: Improving Fairness and Accuracy by Learning to Defer. In *Advances in Neural Information Processing Systems*, vol. 31 <https://doi.org/10.48550/arXiv.1711.06664> (Curran Associates, Inc., 2018).
- Verma, R. & Nalisnick, E. T. Calibrated Learning to Defer with One-vs-All Classifiers. In Chaudhuri, K. *et al.* (eds.) *International Conference on Machine Learning, ICML 2022, 17–23 July 2022, Baltimore, Maryland, USA*, vol. 162 of *Proceedings of Machine Learning Research*, 22184–22202 <https://doi.org/10.48550/arXiv.2202.03673> (PMLR, 2022).
- Verma, R., Barreón, D. & Nalisnick, E. Learning to defer to multiple experts: Consistent surrogate losses, confidence calibration, and conformal ensembles. In *International Conference on Artificial Intelligence and Statistics*, 11415–11434 <https://doi.org/10.48550/arXiv.2210.16955> (PMLR, 2023).
- Hemmer, P., Thede, L., Vössing, M., Jakubik, J. & Kühl, N. Learning to defer with limited expert predictions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 6002–6011 <https://doi.org/10.1609/aaai.v37i5.25742> (2023).
- Wang, X. *et al.* Chestx-ray8: Hospital-scale chest x-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2097–2106 <https://doi.org/10.1109/CVPR.2017.369> (2017).
- Peterson, J. C., Battleday, R. M., Griffiths, T. L. & Russakovsky, O. Human uncertainty makes classification more robust. In *Proceedings of the IEEE/CVF international conference on computer vision*, 9617–9626 <https://doi.org/10.1109/ICCV.2019.00971> (2019).
- Krizhevsky, A. Learning multiple layers of features from tiny images. *University of Toronto* (2012).
- Zhu, Z., Liu, T. & Liu, Y. A second-order approach to learning with instance-dependent label noise. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 10113–10123 <https://doi.org/10.48550/arXiv.2012.11854> (2021).
- Berthon, A., Han, B., Niu, G., Liu, T. & Sugiyama, M. Confidence scores make instance-dependent label-noise learning possible. In *International conference on machine learning*, 825–836 <https://doi.org/10.48550/arXiv.2001.03772> (PMLR, 2021).
- Ke, G. *et al.* LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In Guyon, I. *et al.* (eds.) *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA*, 3146–3154 (2017).
- Jesus, S. *et al.* Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2022* <https://doi.org/10.48550/arXiv.2211.13358> (2022).
- Amarasinghe, K. *et al.* On the importance of application-grounded experimental design for evaluating explainable ml methods. *Proceedings of the AAAI Conference on Artificial Intelligence*, **38**(19), 20921–20929, <https://doi.org/10.1609/aaai.v38i19.30082> (2024).
- Levy, A., Agrawal, M., Satyanarayan, A. & Sontag, D. Assessing the impact of automated suggestions on decision making: Domain experts mediate model errors but take less initiative. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–13 <https://doi.org/10.1145/3411764.3445522> (2021).
- Mitchell, T. J. & Beauchamp, J. J. Bayesian variable selection in linear regression. *Journal of the american statistical association* **83**, 1023–1032, <https://doi.org/10.2307/2290129> (1988).
- Burden, R. L. & Faires, J. D. 2.1 bisection method. In *Numerical analysis*, vol. 3 (PWS Publishers, 1985).
- Gulshan, V. *et al.* Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *Jama* **316**, 2402–2410, <https://doi.org/10.1001/jama.2016.17216> (2016).

26. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M. & Bouchachia, A. A survey on concept drift adaptation. *ACM computing surveys (CSUR)* **46**, 1–37, <https://doi.org/10.1145/2523813> (2014).
27. Elkan, C. The foundations of cost-sensitive learning. In *International joint conference on artificial intelligence*, vol. 17, 973–978 (Lawrence Erlbaum Associates Ltd, 2001).
28. Han, J., Huang, Y., Liu, S. & Towey, K. Artificial intelligence for anti-money laundering: a review and extension. *Digital Finance* **2**, 211–239, <https://doi.org/10.1007/s42521-020-00023-1> (2020).
29. Schwartz-Ziv, R. & Armon, A. Tabular data: Deep learning is not all you need. *Information Fusion* **81**, 84–90, <https://doi.org/10.1016/j.inffus.2021.11.011> (2022).
30. Borisov, V. *et al.* Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems* <https://doi.org/10.1109/TNNLS.2022.3229161> (2022).
31. Alves, J. *et al.* Financial fraud alert review dataset. *figshare* <https://doi.org/10.6084/m9.figshare.28351172>.
32. Grimstad, S. & Jørgensen, M. Inconsistency of expert judgment-based estimates of software development effort. *Journal of Systems and Software* **80**, 1770–1777, <https://doi.org/10.1016/j.jss.2007.03.001> (2007).
33. Remenyi, B. *et al.* Inter-rater and intra-rater reliability and agreement of echocardiographic diagnosis of rheumatic heart disease using the world heart federation evidence-based criteria. *Heart Asia* **11** <https://doi.org/10.1136/heartasia-2019-011233> (2019).
34. Lippens, L., Vermeiren, S. & Baert, S. The state of hiring discrimination: A meta-analysis of (almost) all recent correspondence experiments. *European Economic Review* **151**, 104315, <https://doi.org/10.1016/j.eurocorev.2022.104315> (2023).
35. Mitchell, S., Potash, E., Barocas, S., D'Amour, A. & Lum, K. Algorithmic fairness: Choices, assumptions, and definitions. *Annual Review of Statistics and Its Application* **8**, 141–163, <https://doi.org/10.1146/annurev-statistics-042720-125902> (2021).
36. Corbett-Davies, S., Pierson, E., Feller, A., Goel, S. & Huq, A. Algorithmic decision making and the cost of fairness. In *Proceedings of the 23rd acm sigkdd international conference on knowledge discovery and data mining*, 797–806 <https://doi.org/10.1145/3097983.3098095> (2017).
37. Alves, J. V. *et al.* Cost-sensitive learning to defer to multiple experts with workload constraints. *Transactions on Machine Learning Research*, 2835–8856, <https://doi.org/10.48550/arXiv.2403.06906> (2024).
38. Guo, C., Pleiss, G., Sun, Y. & Weinberger, K. Q. On calibration of modern neural networks. In *International conference on machine learning*, 1321–1330, <https://doi.org/10.48550/arXiv.1706.04599> (PMLR, 2017).
39. Hofmann, H. Statlog (German Credit Data). *UCI Machine Learning Repository* <https://doi.org/10.24432/C5NC77> (1994).
40. Tailor, D., Patra, A., Verma, R., Manggala, P. & Nalisnick, E. Learning to defer to a population: A meta-learning approach. In *International Conference on Artificial Intelligence and Statistics*, 3475–3483, <https://doi.org/10.48550/arXiv.2403.02683> (PMLR, 2024).
41. Akiba, T., *et al.* (eds.) *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4–8, 2019*, 2623–2631, <https://doi.org/10.1145/3292500.3330701> (ACM, 2019).

Author contributions

Conceptualization, J.V.A., D.L., S.J., M.O.P.S., P.S., M.A.T.F.; Data Generation Methodology, J.V.A., D.L., S.J., M.O.P.S., P.S., M.A.T.F.; Final Dataset Generation, J.V.A.; Synthetic expert analysis and validation, J.V.A.; L2D Benchmark Conceptualization, J.V.A., P.S., J.L.; L2D Benchmark Result Analysis, J.V.A.; Code and respective availability, J.V.A., S.J.; Visualization, J.V.A.; Writing, J.V.A., S.J., J.L.; Supervision, P.S., M.A.T.F., M.O.P.S., P.B.; Manuscript Review, J.V.A., S.J., J.L., P.B., M.A.T.F.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to J.V.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025